The CCME Release Guarantees

Main findings, proofs and experimental results (condensed edition)

Semen Fediukovich

September 2026

Technical report accompanying `specs/ccme-spec/SPEC.md` 3.1.0-rc.2
at commit `252216ba` of `github.com/yohimik/dispat`

Abstract

Releasing interdependent packages is a distributed transaction without a coordinator or rollback: each publish is an irreversible write to an independent registry. CCME defines a resumable release algorithm and states eight guarantees: termination, completeness, version stability, exactly-once tag-ledger delivery, retry-invariant non-widening and convergence, channel discharge and one-way axis independence.

This report formalises the algorithm of the 3.1.0 release candidate and proves the guarantees under its explicit retry invariant. The candidate replaces the window test of earlier versions by delivery: a propagated contribution stays owed to a consumer until the source has released it and the consumer has released after that, which closes the orphan a consumer reaches by getting ahead of a failed provider, at the price of an owed window per dependency edge and a refusal (`E201`) where two releases on one commit could hide a debt. The report also proves the determinism and idempotency requirements of SPEC §17.2 and the duality of SPEC §13.7b, and states what the build readiness relation of SPEC §19.2a buys in run duration.

The current departures of the reference implementation from the candidate are tabulated with their status. The final section reports every executed campaign with its pinned binary and environment, including the two measurements of 22 September 2026 on which the execution profile rests: two kernel farm groups that met their public reference times with a second node, twice each after one rejected candidate, and an LLVM matrix released through two workers in less time than on one node, once per side, with a reproduction kit whose binary rebuilds were verified byte for byte and whose machine definitions were planned but not applied. The extended edition, built from the same sources, adds the transcriptions of the distributed and polyrepository profiles, the cost analysis, the historical audit of CCME 1.0.0, the conformance table and the full campaign narratives.

Specification snapshot. The normative subject is `specs/ccme-spec/SPEC.md` as committed at `252216bab9bd4052ba181a2ceb47e21363b8eff3`, stamped 3.1.0-rc.2 and pending release as 3.1.0-rc.3 together with dispat 1.11.0-rc.4; its `DESIGN-HISTORY.md` entries of 21 and 22 September 2026 record what the engine implements. Where the report says *the current text* it means this commit; the released CCME 2.0.0 at `d3ce8ae6` is cited where a change since then matters. A reference whose number carries an F, such as Proposition F73, points into the extended edition (`technical-report-extended.pdf`).

Contents

1 Main findings **4**

1.1 The problem . . . 4
1.2 What this report establishes . . . 4
1.3 Scope, and how to read this edition . . . 5

2 The model **6**
2.1 History . . . 6
2.2 Workspace . . . 6
2.3 Bumps and versions . . . 7
2.4 State . . . 8
2.5 Units . . . 10
2.6 Runs . . . 11

3 Propagation **13**
3.1 The procedure, as written . . . 13
3.2 The traversal . . . 15
3.3 Phase 1: the channel axis . . . 16
3.4 Phase 2: channel resolution . . . 16
3.5 Phase 3: the bump axis . . . 17

4 The publish protocol **19**
4.1 Records under the lock . . . 19
4.2 Tag after, immediately, always, and once . . . 20
4.3 Order . . . 20
4.4 Build readiness . . . 21
4.5 Partial failure . . . 22
4.6 Reconciliation and manifest writes . . . 24

5 Execution: what readiness and placement cost **25**
5.1 Build readiness durations . . . 25

6 The eight guarantees **27**
6.1 Lemmas . . . 28
6.2 G1: termination . . . 30
6.3 G2: completeness . . . 31
6.4 G3: version stability . . . 32
6.5 G4: no double release . . . 33
6.6 G5: no blast-radius widening . . . 34
6.7 G6: convergence . . . 35
6.8 G7: channel discharge . . . 36
6.9 G8: axis independence . . . 37

7 Determinism, idempotency, resumption **37**

8 The staleness dual **39**

9 Departures of the implementation and boundaries of the text **42**

10 Evaluation: the executed campaigns **44**

10.1 Evidence boundaries and units 44

10.2 RQ1: the consumer-window mechanism 45

10.3 RQ2: retry and identity stability 46

10.4 RQ3: availability boundaries and CI 48

10.5 RQ4: planning cost 49

10.6 RQ5: distributed execution 49

10.6.1 The kernel farm 50

10.6.2 The LLVM pair 50

10.7 Threats to validity 53

10.8 Reproduction 53

A Notation **53**

B The conference abstract, read against this report **54**

1 Main findings

1.1 The problem

A workspace is a repository holding many independently versioned packages that depend on one another. Releasing it means choosing a new version for each package that needs one, writing those versions into manifests, uploading artefacts to registries, and recording what happened. The recording is the hard part: registries have no transactions, a publish is an irreversible write to a service the release tool does not control, several registries may be involved, and no coordinator can commit or abort them as a unit. A run that publishes seven packages is seven independent, individually fallible writes.

CCME[1] answers this the way a saga does. The release plan is a pure function of the repository at a fixed commit; each completed leg is recorded by an annotated git tag written immediately after that leg succeeds; and recovery is re-execution: run the engine again and it recomputes the same transaction, skipping the legs already recorded. That design works only if re-execution is well behaved, and SPEC §13.7c states the eight properties that say it is:

G1 termination
: propagation halts, whatever the declared depth.

G2 completeness
: a contribution admitted to a dependent survives in every later run until that dependent releases at or after a release of every owed source carrying it; there are no orphans, whether the provider failed or the consumer got ahead of it.

G3 version stability
: a package released late is released at the version it was originally planned at.

G4 no double ledger delivery
: a contribution does not survive its delivery in the tag ledger; this says nothing unconditional about external effects.

G5 no blast-radius widening
: under the retry invariant, later targets are a subset of the first run's.

G6 restricted convergence
: under the retry invariant, repeated successful progress empties the initial plan within as many runs as it had publishable packages.

G7 channel discharge
: a prerelease channel is proposed to a package only until it is released on it.

G8 one-way axis independence
: suppressing a version bump does not suppress a channel change; resolved channels may suppress bumps.

1.2 What this report establishes

The report writes the algorithm down once, formally (Sections 2 to 4), proves the guarantees inside that model (Section 6), and reports what the reference implementation was observed to do in the campaigns that were executed (Section 10). The findings, each with the result that carries it:

[1] *Conventional Commits: Monorepo Extension*, version 3.1.0-rc.2 at commit `252216ba`, `specs/ccme-spec/SPEC.md` in the `dispat` repository. Referred to throughout as SPEC, cited by section number.

1. **The eight guarantees hold for the model under the explicit retry invariant of spec §13.7c and two hypotheses the specification does not state**, (R) that owed sources reach their target and (K) that propagation edges order and block (Theorems 72, 74, 76, 79, 81, 85, 89 and 91). (R) holds outright for a unit with a single source; it can fail for a multi-source unit, and the reference engine exhibited exactly that failure until a fix that postdates the measured builds (proof obligation F132).
2. **Delivery closes the orphan a consumer reaches by getting ahead of a failed provider.** The candidate admits a propagated contribution until its source has delivered it (SPEC §13.4a); the model transcribes this test (Definition 10) and Theorem 74 is proved for it. Two rules follow and are stated as boundaries rather than proved away: an owed window per dependency edge and the refusal `E201`, which the specification states and the engine does not yet implement (proof obligation F131), and the same-commit case that the protocol reports rather than resolves (proof obligation F133).
3. **The plan is a deterministic, idempotent function of the fixed inputs, and a run after a partial failure plans the failed packages, the blocked ones and the consumers still owed** (Theorem 93, Corollary 95, Theorem 94).
4. **The staleness audit of spec §13.7b never over-reports and under-reports in three shapes**, all with a multi-source unit; the second shape is the engine defect above (Theorem 98, Remark 99).
5. **Build readiness buys wall time in a form that can be stated.** With unlimited slots a run lasts its longest path and a weaker relation never lengthens it; the chain, fan-out and fan-in formulas are exact; under slot budgets a greedy schedule is within the list-scheduling bound and can show Graham's anomaly (Section 5.1). The placement invariance of the distributed profile, that worker count, placement and completion order do not change the plan, is stated and proved in the extended edition (Proposition F67).
6. **The implementation departs from the text in eight recorded places**, none of which loses a release silently except the stranded consumer of proof obligation F131; Section 9 tabulates them with their status.
7. **The measurements.** In every state of a finite projection and in every completed episode, the consumer-relative window retained exactly the residual obligations, and a source-relative window would have lost obligations in 84 of the 120 designed fault episodes and in 260 of 772 states (Section 10.2). Recorded progress survived repeated failure without version drift in 130 of 130 rows, and the restricted oracle agreed on all 300 histories of each companion campaign (Section 10.3). A configured barrier distinguished local build availability from required publication, and both a failed and an absent required test blocked release (Section 10.4). Planning cost rose from 0.017 s at 100 packages to about 19 s at 10,000 on a capped host (Section 10.5). On a kernel build farm, two groups met their public reference times with a second node, twice each after one rejected candidate, where the same node alone did not; on an LLVM matrix, a release through two workers took 27:48.94 against 41:12.02 on one node, once per side; a one-heavy-package fixture showed the predicted negative case (Section 10.6). No speedup of the profile or of the engine is claimed beyond what these pairs measured.

1.3 Scope, and how to read this edition

The subject is the release computation of SPEC §13 and the publish protocol of SPEC §19, with the execution rules of SPEC §§19.2a, 28 as transcribed obligations. Message syntax (SPEC §4–SPEC §8) is taken as given: units arrive already parsed. Outside the theorems, and not spoken for by them, are manifest range reconciliation (SPEC §9.4, SPEC §19.5), the diagnostic registry

of SPEC §16 beyond the codes used as labels, executor scheduling beyond the stated relations and bounds, shared-version groups (SPEC §13.9a), and the adapter and rollback protocols of SPEC §§25, 26. The report claims no novelty for sagas, idempotent consumers, breadth-first search or list scheduling; its contribution is the formal model, corrected hypotheses, counterexamples, the ledger of departures, and the record of what was measured.

This is the condensed edition. The extended edition, distributed beside it as `technical-report-extended.pdf` and built from the same sources, adds the transcriptions of the distributed profile (SPEC §28) and of the repository-local windows (SPEC §27), the cost analysis of SPEC §§13.11, 20.7, the historical audit of CCME 1.0.0 with its six counterexamples, the conformance table that maps every result to the specification's vectors and to the integration test behind it, and the full narratives of the campaigns, including every discarded attempt. A reference whose number carries an F, such as *Proposition F73*, points into the extended edition; every other reference is to this document. Appendix A is the symbol table, and Appendix B reads the companion conference abstract against this report sentence by sentence. SPEC is licensed GPL-3.0-or-later and this report reproduces its definitions and pseudocode; it is distributed under the same licence (`paper/LICENSE`).

2 The model

Everything in this report is stated over the objects defined here. Each definition transcribes a numbered section of SPEC and cites it. Where the transcription makes a choice the source leaves implicit, the choice is flagged in a remark rather than buried.

2.1 History

Definition 1 (commit history, SPEC §12.2)**.** The history is a finite directed acyclic graph on a set of commits $\mathcal{K}$, with an edge from a commit to each of its parents. A distinguished commit $h \in \mathcal{K}$ is *HEAD*. For $c \in \mathcal{K}$, $\mathrm{reach}(c) \subseteq \mathcal{K}$ is the set of commits reachable from c by following parent edges, including c itself. Write $c' \preceq c$ for $c' \in \mathrm{reach}(c)$ ("c' is an ancestor-or-self of c").

h is fixed for the whole report. The retry and convergence guarantees of SPEC §13.7c are stated at a fixed h; advancing it may legitimately add, remove, or redirect work. Structural termination and single-state determinism do not need that restriction.

Remark 2. Ancestry, not commit date or log order, is the only ordering used. This is what makes cancellation deterministic under merges and rebases (SPEC §10.4) and what lets the model ignore clocks entirely. One consequence is used without comment below: since h reaches every commit considered and the graph is acyclic, $h \preceq c$ holds for no $c \neq h$.

2.2 Workspace

Definition 3 (workspace graph, SPEC §13.1, SPEC §8.4)**.** $\mathcal{P}$ is the finite set of packages present at h. For a fixed set of edge kinds K_{prop} (the configured `propagation.kinds`), $E \subseteq \mathcal{P} \times \mathcal{P}$ contains (a, b) when a declares a dependency on b of a kind in K_{prop}. The *dependents* of p are $\mathrm{dep}(p) = \{d : (d, p) \in E\}$. The graph $(\mathcal{P}, E)$ is acyclic. Two further kind sets, K_{order} (`publish.orderKinds`) and K_{block} (`publish.blockingKinds`), select the edges of the publish order and of failure blocking (SPEC §19.2, SPEC §19.3); by default K_{prop} and K_{order} contain `optionalDependencies` and K_{block} does not (SPEC §14).

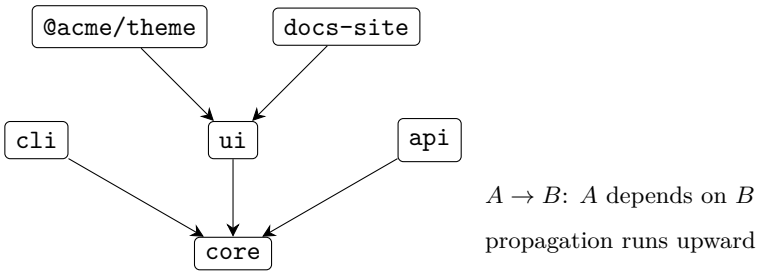


Figure 1: The running workspace of Example 4. Distances from $S = \{\texttt{core}\}$ are $\delta(S, \texttt{cli}) = \delta(S, \texttt{ui}) = \delta(S, \texttt{api}) = 1$ and $\delta(S, \texttt{docs-site}) = \delta(S, \texttt{@acme/theme}) = 2$.

Acyclicity is an assumption the model may make because the engine enforces it: a cycle over the propagation or publish-ordering kinds is the repository-scoped error `E200` and aborts the run before any plan is computed. SPEC §13.1 justifies rejecting rather than accommodating cycles: no publish order exists for a cycle, so whichever member goes first declares a range on a version that does not yet exist, and if the run then fails that state is permanent.

Propagation travels *against* the dependency edges, from a changed package to the packages that consume it; that is, along $\mathrm{dep}(\cdot)$.

Example 4 (the running workspace, SPEC §22)**.** The running examples use the six-package workspace shared by many conformance vectors:

$$\mathcal{P} = \{\, \texttt{core},\ \texttt{cli},\ \texttt{ui},\ \texttt{api},\ \texttt{docs-site},\ \texttt{@acme/theme} \,\},$$

$$E = \{\, (\texttt{cli}, \texttt{core}),\ (\texttt{ui}, \texttt{core}),\ (\texttt{api}, \texttt{core}),\ (\texttt{docs-site}, \texttt{ui}),\ (\texttt{@acme/theme}, \texttt{ui}) \,\},$$

drawn in Figure 1, with the initial tag state

$$\sigma_0 = \{\ \texttt{core@1.4.2},\ \texttt{cli@2.0.0},\ \texttt{ui@0.9.1},\ \texttt{api@1.2.0},\ \texttt{@acme/theme@1.0.0}\ \}.$$

`docs-site` has never been released, so it has no baseline; it is private and publishes to an internal registry, which by SPEC §13.10a exempts it from nothing.

Definition 5 (distance, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $d \in \mathcal{P}$, $\delta(S, d)$ is the least n for which there are $p_0 \in S$ and $p_1, \ldots, p_n = d$ with $p_{i+1} \in \mathrm{dep}(p_i)$ for all i, and ∞ if no such chain exists. Note $\delta(S, d) = 0$ exactly when $d \in S$. We say p *reaches* d when $\delta(\{p\}, d) < \infty$; equivalently, d depends on p transitively over K_{prop}.

2.3 Bumps and versions

Definition 6 (bumps, SPEC §7.1, SPEC §9.1)**.** $\mathbb{B} = \{\mathsf{none}, \mathsf{patch}, \mathsf{minor}, \mathsf{major}\}$ ordered $\mathsf{none} < \mathsf{patch} < \mathsf{minor} < \mathsf{major}$. Being a finite chain, $(\mathbb{B}, \leq)$ is a join-semilattice with least element none and join $\sqcup = \max$.

The whole of SPEC §9.1's "bumps accumulate" is the statement that the combining operator is this join. Two consequences that SPEC states separately are then immediate and are used without further comment: combination is associative, commutative and idempotent, so the order in which contributions are gathered cannot matter; and $x \sqcup y \geq x$, so propagation can only ever raise a package's bump, never lower or replace a direct decision.

Definition 7 (versions and channels, SPEC §11.1–SPEC §11.4)**.** $\mathcal{V}$ is the set of SemVer 2.0.0 versions with SemVer precedence. $\mathcal{C}h = \{\mathsf{stable}\} \cup \mathcal{N}$ for a set $\mathcal{N}$ of channel names. $\mathrm{ch}(v) = \mathsf{stable}$ if v carries no prerelease identifier, and otherwise the first (alphanumeric) prerelease identifier of v. $\mathrm{core}(v)$ is the `major.minor.patch` triple of v and $\mathrm{cnt}(v)$ the numeric counter of a prerelease. $\mathrm{apply}(v, b)$ applies bump b to v, with $\mathrm{apply}(v, \mathsf{none}) = v$.

2.4 State

Everything above is fixed by h and the configuration. Exactly one thing changes from run to run, and isolating it is what makes the guarantees provable as statements about a sequence.

Definition 8 (engine state, SPEC §13.2, SPEC §12.3)**.** A state σ is a finite set of release tags $(p, v, c) \in \mathcal{P} \times \mathcal{V} \times \mathcal{K}$ with $c \preceq h$. States are ordered by inclusion, $\sigma \leq \sigma'$ iff $\sigma \subseteq \sigma'$. From σ:

- $T_\sigma(p) = \{\, c : (p, v, c) \in \sigma \,\}$ is the set of *release commits* of p;
- $\mathrm{base}_\sigma(p)$ is the highest-precedence version among the tags of p in σ, undefined if there are none;
- $\mathrm{basec}_\sigma(p)$ is the commit of that tag;
- $\mathrm{sbase}_\sigma(p)$ is the highest-precedence non-prerelease version among them, and $\mathrm{scom}_\sigma(p)$ its commit.

Where $\mathrm{base}_\sigma(p)$ is undefined, $\mathrm{ch}(\mathrm{base}_\sigma(p)) = \mathsf{stable}$ by convention (SPEC §11.1): a never-released package is graduated by nothing and is put on a train by any directive naming it. We say p is *tagged at* h in σ when $h \in T_\sigma(p)$.

The baseline functions additionally assume a unique highest-precedence reachable tag for each package and each stable baseline. States with tied precedence and different commits need a specified tie rule and are outside this proof's domain. Operational lock and temporary refs are not release tags and are not members of σ. In every state considered here a tag at h, where there is one, is the baseline: it was written by a run at h (Definition 25) at a version exceeding the baseline of that run (`E195`), so $h \in T_\sigma(p)$ implies $\mathrm{basec}_\sigma(p) = h$.

That release-tag states only ever grow is not an assumption about tidy operators; it is normative. SPEC §19.1 forbids moving or deleting a pushed tag and SPEC §12.4 makes tags authoritative over manifests, because a tag is the record that a leg of the transaction completed and an unwritten-back or rewritten record is exactly what makes a saga unrecoverable.

Definition 9 (train and undelivered windows, SPEC §13.3)**.**

$$W_\sigma(p) \;=\; \mathrm{reach}(h) \setminus \mathrm{reach}(\mathrm{scom}_\sigma(p)), \tag{1}$$

and $W_\sigma(p) = \mathrm{reach}(h)$ when p has no stable baseline in σ. The undelivered window is

$$W^\circ_\sigma(p) = W_\sigma(p) \setminus \mathrm{reach}(\mathrm{basec}_\sigma(p)),$$

with an empty subtracted set when p has no baseline tag.

$W_\sigma(p)$ is the train window used to aggregate a prerelease target from its stable base. $W^\circ_\sigma(p)$ is the fresh-admission window for direct work, corrections, channels, and propagated channels to dependent p. For a stable baseline they coincide; on a train they must not be conflated.

Definition 10 (delivery, SPEC §13.4a)**.** For packages P, D and a commit C,

$$\mathrm{dlv}_\sigma(P, C, D) \;\equiv\; \exists\, t \in T_\sigma(P) :\; C \preceq t \;\wedge\; t \preceq \mathrm{basec}_\sigma(D),$$

and $\mathrm{dlv}_\sigma(P, C, D)$ is false when D has no baseline. Read: D has released at or after a release of P that carries C.

Delivery is a relation between two packages' release positions, decided by ancestry alone. Releasing past a commit is not delivery: a D whose baseline reaches C but reaches no release of P carrying C has not had C delivered by P. Only the source's release, followed by the target's, is (SPEC §13.4a).

Definition 11 (owed window, SPEC §13.3)**.** For every edge $(D, P) \in E$ where D has a baseline,

$$W^{\mathrm{ow}}_{\sigma}(P, D) \;=\; \mathrm{reach}(h) \setminus \bigcup \{\, \mathrm{reach}(t) \;:\; t \in T_{\sigma}(P),\; t \preceq \mathrm{basec}_{\sigma}(D) \,\},$$

the commits reachable from h and from no release commit of P that $\mathrm{basec}_{\sigma}(D)$ reaches.

Lemma 12 (the owed window is the undelivered set)**.** *For $(D, P) \in E$ with D having a baseline, and every $c \preceq h$, $c \in W^{\mathrm{ow}}_{\sigma}(P, D)$ iff $\neg \mathrm{dlv}_{\sigma}(P, c, D)$.*

Proof. $c \notin W^{\mathrm{ow}}_{\sigma}(P, D)$ iff $c \in \mathrm{reach}(t)$ for some $t \in T_{\sigma}(P)$ with $t \preceq \mathrm{basec}_{\sigma}(D)$, which is Definition 10 verbatim. □

SPEC §13.3 adds the owed windows to the union of windows that SPEC §13.4 parses, so that a debt D is owed stays *visible* after D has released past the commit and P has released it: once P has, no window of P holds the commit, and if D released past it before P did, no window of D does either. The window makes a debt visible; whether it is owed is Definition 19's question, and the window admits nothing by itself.

Remark 13 (two cases SPEC §13.3 describes, and one it does not). SPEC §13.3 says that where the newest release of P that $\mathrm{basec}_{\sigma}(D)$ reaches is P's own baseline, nothing is added. That is right: then $\mathrm{reach}(\mathrm{basec}_{\sigma}(P))$ is among the subtracted reaches, so $W^{\mathrm{ow}}_{\sigma}(P, D) \subseteq W^{\circ}_{\sigma}(P) \subseteq W_{\sigma}(P)$, and this is the position of every consumer that never got ahead of its provider. It also says that where $\mathrm{basec}_{\sigma}(D)$ reaches no release of P nothing is added, because D released before P's first release and $W_{\sigma}(D)$ already holds everything since. Definition 11 read literally gives $W^{\mathrm{ow}}_{\sigma}(P, D) = \mathrm{reach}(h)$ in that case, which is not "nothing"; and the prose's reason covers only commits after D's release, while a commit $c \preceq \mathrm{basec}_{\sigma}(D)$ that P first released at a commit $\mathrm{basec}_{\sigma}(D)$ does not reach is, by Definition 10, undelivered and owed, and lies in neither $W_{\sigma}(D)$ (for a stable D) nor $W_{\sigma}(P)$. The model follows the formula, which is the reading the guarantees need; the two sentences of SPEC §13.3 need reconciling, and that is reported as a specification obligation rather than resolved here.

Example 14 (windows)**.** Let the history be $c_0 \to c_1 \to c_2 = h$ with σ_0 of Example 4, where the tags sit at c_0. Then

$$W_{\sigma_0}(\texttt{core}) = \{c_1, c_2\}, \qquad W_{\sigma_0}(\texttt{docs-site}) = \{c_0, c_1, c_2\},$$

the second because a package with no stable baseline has the whole reachable history pending: SPEC §12.5 makes an unreleased package's first release account for everything, which is also why adopting the engine on an existing repository does not need a migration step. The owed window $W^{\mathrm{ow}}_{\sigma_0}(\texttt{core}, \texttt{cli})$ subtracts $\mathrm{reach}(c_0)$, the one `core` release that $\mathrm{basec}_{\sigma_0}(\texttt{cli}) = c_0$ reaches, and equals $W_{\sigma_0}(\texttt{core})$: nothing is added for a consumer level with its provider. Now let $\sigma_1 = \sigma_0 \cup \{\texttt{cli@2.1.0} \text{ at } c_2\}$, the state after `cli` has proceeded past c_2 on a cause of its own while `core` failed (Example 39). Then $W^{\circ}_{\sigma_1}(\texttt{cli}) = \emptyset$, $W_{\sigma_1}(\texttt{cli}) = \emptyset$, and still $W^{\mathrm{ow}}_{\sigma_1}(\texttt{core}, \texttt{cli}) = \mathrm{reach}(c_2) \setminus \mathrm{reach}(c_0) = \{c_1, c_2\}$, because $\mathrm{basec}_{\sigma_1}(\texttt{cli}) = c_2$ reaches no `core` release newer than c_0. If `core` then releases at c_2, $W^{\mathrm{ow}}(,)$ becomes empty as well: `core` has delivered c_2 to `cli`, or the ledger reads as if it had (Proposition 26).

Remark 15 (the window is measured from the last *stable* tag). This is deliberate and it serves SPEC §11.4: a package on a prerelease train must accumulate every commit since its last *stable* release, because the train's target version is recomputed from the stable baseline on every run so that a breaking change arriving mid-train moves the whole train. The consequence, which Section F10 returns to at length, is that a commit already published in a prerelease of p is still a member of $W_{\sigma}(p)$.

2.5 Units

Definition 16 (units, SPEC §4.4, SPEC §13.3, SPEC §13.4, SPEC §13.4b)**.** $\mathcal{U}$ is the set of units carried by commits in

$$\bigcup_{p \in \mathcal{P}} W_\sigma(p) \;\cup \bigcup_{(D,P) \in E,\ D \text{ has a baseline}} W^{\mathrm{ow}}_\sigma(P, D),$$

after parsing (SPEC §20), scope resolution (SPEC §6) and the application of corrections (SPEC §13.4b). For $u \in \mathcal{U}$: $c_u \in \mathcal{K}$ is its commit; $\mathrm{scope}(u) \subseteq \mathcal{P}$ its resolved scope-set; $\mathrm{bump}(u) \in \mathbb{B}$ the bump its type and `!` map to; and its directives are

$$\mathrm{pb}(u) \in \mathbb{B}, \quad \mathrm{pd}(u) \in \mathbb{N} \cup \{\infty\}, \quad \mathrm{ps}(u) \subseteq \mathcal{P}, \quad \mathrm{pc}(u), \quad \mathrm{pcd}(u), \quad \mathrm{pcs}(u) \subseteq \mathcal{P},$$

where $\mathrm{pb}(u)$ is the *effective* propagated bump: the value the unit states, or $\mathrm{bump}(u)$ where it states `inherit`. The two axes default differently and the asymmetry is deliberate (SPEC §8.1). `Propagate` defaults to `patch`, not to `inherit`: a dependent reached by propagation has had a dependency change under it but no change to its own public API, so `patch` is the honest signal, and `feat(core)^` therefore gives its consumers a patch rather than a minor. `Propagate-Channel` defaults to `inherit`, since a channel is not a quantity that can be understated. Both defaults apply only once the corresponding depth is non-zero, and both depths default to 0 (SPEC §8.3, SPEC §8.3a): a unit that says nothing releases its own packages and nothing else. $\mathrm{pc}(u)$ is a channel value: a name, `stable`, `inherit`, `none`, or a transition $\langle f \to t \rangle$.

Taking corrections as already applied is what SPEC §13.4b prescribes: it requires `Edits` and `Deletes` to be resolved into the tuple stream before cancellation runs, and notes that every later phase operates on the corrected stream, which downstream phases cannot distinguish from one that was authored that way.

Remark 17 (the corrected stream depends on σ)*.* It does not follow, and SPEC §13.4b does not claim, that the stream is independent of the state. A correction reaches only *undischarged* work: it applies to a (package, unit) pair only while that package's window still contains the target commit, and once the package has released the target the correction is a no-op for it (`W209`, SPEC §7.4.2). So $\mathcal{U}$ is properly a function $\mathcal{U}_\sigma$, and a record suppressed by a `Deletes` for a package returns to the stream for that package once the package releases. This matters in exactly one place, and it is the same place that a lapsed cancellation matters: proof obligation F140. Everywhere else the two readings coincide, because a released package's own window is empty and its direct contribution is therefore already spent, so the notation $\mathcal{U}$ is kept unindexed below. The retry invariant of Section 6 fixes the corrected stream outright, which also fixes the parsed set.

Remark 18 (what the owed windows make visible, and what they do not)*.* The owed windows of Definition 16 range over the *direct* edges of E. A debt, by Definition 10, is between a unit's source and any target the unit reaches, at any depth. For a target at distance 1 the two coincide: if P owes D and $(D, P) \in E$, then $c_u \in W^{\mathrm{ow}}_\sigma(P, D)$ by Lemma 12, and the unit is parsed. For a target at distance 2 or more it need not: with $D \to M \to P$, the commit c_u is parsed for D's sake only through $W()$ of some package or through $W^{\mathrm{ow}}(P, M)$ and $W^{\mathrm{ow}}(M, D)$, and once M has been delivered c_u by P and D has been delivered c_u by M (which says nothing about P's release reaching D), every one of those windows can be empty while $\mathrm{dlv}_\sigma(P, c_u, D)$ is still false. The model transcribes SPEC §13.3 as written; the theorems of Section 6 fix the parsed stream by the retry invariant and are unaffected. The gap is reported as a specification obligation rather than repaired here.

Definition 19 (suppression, source packages and owed sources, SPEC §13.4a)**.** $\mathrm{canc}(c, p)$ holds when c is an ancestor-or-self of some `cancel` commit whose resolved scope contains

p (SPEC §10.3); held(p) holds when p's effective `Release-As` directive is `none` (SPEC §13.6a). Both are computed from the corrected stream, h, configuration, and release-tag state. They are constant across retries only where the retry invariant says those inputs remain fixed. Write

$$\mathrm{disch}_\sigma(p, c) \;\equiv\; c \notin W^\circ_\sigma(p),$$

and then

$$\mathrm{src}_\sigma(u) \;=\; \{\, p \in \mathrm{scope}(u) \;:\; \mathrm{disch}_\sigma(p, c_u) \;\vee\; \neg(\mathrm{canc}(c_u, p) \vee \mathrm{held}(p)) \,\}.$$

For a target D, the *owed sources* of u are

$$\mathrm{owed}_\sigma(u, D) \;=\; \{\, P \in \mathrm{src}_\sigma(u) \;:\; \neg\mathrm{dlv}_\sigma(P, c_u, D) \,\},$$

and P *still owes* D, written $\mathrm{owes}_\sigma(P, D)$, when $P \in \mathrm{owed}_\sigma(u, D)$ for some unit u with $\mathrm{adm}_\sigma(u, D)$ (Definition 37).

The disjunction in src() is the normative content of SPEC §13.4a: suppression applies only to *undischarged* work. Once p has published the version carrying u, the artefact its consumers are owed is public, and neither a later `cancel` (a no-op, `W170`) nor a later hold may retroactively strand those consumers. Proof obligation F140 shows that this clause, and not the traversal, is where **G5** is decided. The owed set is the other half of the same section: on the bump axis a unit propagates to D while $\mathrm{owed}_\sigma(u, D)$ is non-empty, and D's provenance names exactly the owed sources.

Lemma 20 (delivery implies discharge at the target)**.** *If* $\mathrm{dlv}_\sigma(P, c, D)$ *then* $c \notin W^\circ_\sigma(D)$. *Consequently, if* $c_u \in W^\circ_\sigma(D)$ *then* $\mathrm{owed}_\sigma(u, D) = \mathrm{src}_\sigma(u)$.

Proof. Delivery gives t with $c \preceq t \preceq \mathrm{basec}_\sigma(D)$, so $c \in \mathrm{reach}(\mathrm{basec}_\sigma(D))$, which Definition 9 subtracts from $W^\circ_\sigma(D)$. For the consequence, no source of u has delivered c_u to D, so none is removed from $\mathrm{src}_\sigma(u)$. □

So for a target that has not released past the commit, the owed set is the whole source set and SPEC §13.4a's delivery test reduces to the window test of CCME 2.0.0. The finer question arises only for a target that released past c_u before its source did: a consumer that proceeded on a cause of its own while the source failed (SPEC §19.3), or one released while the source was held (SPEC §13.6a). Under the polyrepository profile the comparison of Definition 10 is made in P's repository (SPEC §27); that profile is outside this report.

2.6 Runs

Definition 21 (plan)**.** Given σ, the release computation of SPEC §13 yields the bumps $\mathrm{dir}_\sigma(p)$ and $\mathrm{prp}_\sigma(p)$ (Definition 38), the channels $\mathrm{chan}_\sigma(p)$ (Definition 34), and $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$. The plan is

$$\begin{aligned}\Pi(\sigma) \;=\; \{\, p \in \mathcal{P} \;:\; &\neg\mathrm{held}(p) \;\wedge\; (\mathrm{eff}_\sigma(p) \neq \mathsf{none} \\ &\vee\; \mathrm{chan}_\sigma(p) \neq \mathrm{ch}(\mathrm{base}_\sigma(p)) \;\vee\; p \text{ has an applicable exact } \texttt{Release-As} \\ &\qquad\qquad \text{in } W^\circ_\sigma(p)) \,\},\end{aligned}$$

each $p \in \Pi(\sigma)$ carrying the version $\nu_\sigma(p)$ of SPEC §13.9. The fresh bumps above decide whether work is newly admissible; they are not by themselves the prerelease target. On a train, the intended calculation aggregates the surviving corrected units in $W_\sigma(p)$ from the stable baseline, including work already delivered to an earlier prerelease. For graduation, the model applies the specific rule of SPEC §11.5: publish that same train target without a prerelease suffix. The generic summary in SPEC §13.9 is inconsistent with this rule; proof obligation F134 records the textual obligation.

Of the disjuncts, the direct bump, the channel change and the exact version all read $W^{\circ}_{\sigma}(p)$, and are therefore absent for a package tagged at h; only $\mathrm{prp}_{\sigma}(p)$ can put such a package in the plan (Lemma 71). That asymmetry is what SPEC §19.3's "cause of its own" is about.

Remark 22 ($\Pi(\sigma)$ is the released set, not the printed one). SPEC §13.9 computes a would-be version for a held package too, and SPEC §13.6a requires the engine to report it (`W154`) so that the number needed to lift the hold is available without hand computation. That reported entry is part of what an operator calls "the plan", but it is not a release: SPEC §13.10 excludes it. $\Pi(\sigma)$ above is the released set, which is why Theorem 85 needs no exception for holds and why the convergence check of SPEC §19.6 states one: the two are reading the same word differently, and only SPEC §19.6's reading has to be told about held packages.

Definition 23 (a cause of its own, SPEC §19.3)**.** For $p \in \Pi(\sigma)$ and a set $X \subseteq \mathcal{P}$ of packages that have failed or been blocked earlier in the run, p has a *cause of its own* relative to X, written $\mathrm{own}_{\sigma}(p, X)$, when at least one of the following holds: $\mathrm{dir}_{\sigma}(p) \neq$ `none`; $\mathrm{chan}_{\sigma}(p) \neq \mathrm{ch}(\mathrm{base}_{\sigma}(p))$; p has an applicable exact `Release-As` in $W^{\circ}_{\sigma}(p)$; or there is a unit u with $\mathrm{adm}_{\sigma}(u, p)$ and $\mathrm{owed}_{\sigma}(u, p) \not\subseteq X$, a contribution from a provider that is neither failed nor blocked, including one that published in an earlier run.

SPEC §19.3 lists a fresh direct bump, a channel change and a contribution from an unfailed provider; the model adds the exact-version disjunct of Definition 21, which the section does not name, because a package pinned by `Release-As` has a cause that is its own by any reading. The addition is flagged for the specification. The reference implementation follows the section as written: its test of a cause of its own is a fresh unit with a bump other than `none`, a channel change, or an update from a provider that did not fail, so a consumer whose only cause is an exact `Release-As` is blocked behind a failed provider there, and the theorems below read the engine with the disjunct removed.

Definition 24 (well-formed state, SPEC §19.3, `E201`)**.** $\mathrm{wf}(\sigma)$ holds when for every $P \in \Pi(\sigma)$ and every D with $\mathrm{basec}_{\sigma}(D) = h$ and $\mathrm{owes}_{\sigma}(P, D)$: $D \in \Pi(\sigma)$ and P precedes D in the publish order of Definition 44.

A run from a state that is not well-formed would release a package at the baseline commit of a consumer it still owes without that consumer releasing after it. Two releases on one commit have no order (SPEC §13.7b), so afterwards nothing could tell that the consumer came first: it would read as served and stay on the old version with nothing left to detect it. SPEC §19.3 therefore refuses such a run with `E201` before any publication, naming the pair; the remedy is to release the consumer in the same run, or the provider at a later commit, where the debt stays visible (Definition 11).

Definition 25 (run, SPEC §19.1–SPEC §19.3)**.** A run from σ is parameterised by a set $F \subseteq \Pi(\sigma)$ of publishes that fail. If $\neg\mathrm{wf}(\sigma)$ the run is refused: $\mathrm{run}(\sigma, F) = \sigma$. Otherwise the plan is processed in the publish order of Definition 44. Let X_p be the set of packages before p in that order that failed or were blocked. Then p is *blocked* (`W194`) when it transitively depends over K_{block}, in the full workspace graph, on a member of X_p and $\neg\mathrm{own}_{\sigma}(p, X_p)$; it *fails* when it is not blocked and $p \in F$; and it is *published* otherwise. Write $B(\sigma, F)$ for the blocked set and $S(\sigma, F) = \Pi(\sigma) \setminus (F \cup B(\sigma, F))$ for the published set. The run yields

$$\mathrm{run}(\sigma, F) \;=\; \sigma \;\cup\; \{\, (p, \nu_{\sigma}(p), h) \;:\; p \in S(\sigma, F) \,\}.$$

The run *reports* `E201` *for* D when $h \in T_{\sigma}(D)$, $D \in F$, and some $P \in S(\sigma, F)$ has $\mathrm{owes}_{\sigma}(P, D)$.

Four features of this definition carry weight later.

F is chosen adversarially and independently on each run. The guarantees are claims about *every* failure schedule, not about a well-behaved one; a proof that appeals to what usually fails proves nothing about a system whose purpose is to survive what does not usually happen.

Every tag is written at h, never at the commit the work arrived in (SPEC §19.1). This is what makes Lemma 65 a two-case statement rather than an argument about which commits a tag covers, and what makes Lemma 67 a statement about one commit.

A package is tagged if it is released, whatever its publish target, including targets of `none` that upload nothing (SPEC §13.10a). Proposition 88 shows this is forced: the alternative is a permanent hole in **G6**.

Blocking is not the transitive closure of failure. A dependent with a cause of its own *proceeds*, its manifests reconciled to what its providers have published by then (SPEC §19.5), and what the failed provider owed it stays owed and arrives as a catch-up when that provider publishes. This is the consumer-ahead shape of SPEC §13.7a, and it is why $S(\sigma, F)$ may contain a package that was already tagged at h.

Proposition 26 (the one loss, SPEC §19.3)**.** *Let* $\mathrm{wf}(\sigma)$, $\sigma' = \mathrm{run}(\sigma, F)$, *and let the run report* `E201` *for* D *because of* $P \in S(\sigma, F)$. *Then for every unit* u, $P \notin \mathrm{owed}_{\sigma'}(u, D)$, *although* D *published nothing after* P. *If* $\mathrm{owed}_{\sigma'}(u, D) = \emptyset$ *for every* u *with* $\mathrm{adm}_{\sigma}(u, D)$, *then* $D \notin \Pi(\sigma')$, *and no state reachable from* σ' *by runs at this or any later* h *plans the contribution again.*

Proof. D was not released, so $\mathrm{basec}_{\sigma'}(D) = \mathrm{basec}_{\sigma}(D) = h$, and $h \in T_{\sigma'}(P)$. For any u, $c_u \preceq h \preceq \mathrm{basec}_{\sigma'}(D)$ with $t = h$, so $\mathrm{dlv}_{\sigma'}(P, c_u, D)$ and P is not owed. If every admitted unit's owed set is now empty, $\mathrm{prp}_{\sigma'}(D) = \mathsf{none}$ (Definition 38); $W^{\circ}_{\sigma'}(D) = \emptyset$ since $\mathrm{basec}_{\sigma'}(D) = h$, so no other disjunct of Definition 21 holds. A later state adds tags, which keeps the witness $t = h$, and a later h changes neither $T(P)$ nor $\mathrm{basec}(D)$ while D is unreleased; so $\mathrm{dlv}(P, c_u, D)$ persists and the owed set stays empty until D itself releases, which nothing in the ledger now asks for. □

This is the one place where the ledger loses an obligation. The run reports it with its remedy, an explicit `Release-As` on the consumer at a later commit (SPEC §8.6), because no later plan can compute the debt: two tags on one commit have no order. Everywhere else the protocol refuses before the fact (Definition 24) or catches up after it (Theorem 74). The theorems of Section 6 state where they exclude this event.

Definition 27 (run sequence)**.** A *run sequence* at h is $\sigma_1 \le \sigma_2 \le \cdots$ with $\mathrm{wf}(\sigma_1)$ and $\sigma_{k+1} = \mathrm{run}(\sigma_k, F_k)$ for some $F_k \subseteq \Pi(\sigma_k)$. By Definition 25 it is monotone: $\sigma_k \subseteq \sigma_{k+1}$, and $T_{\sigma_{k+1}}(p) \setminus T_{\sigma_k}(p) \subseteq \{h\}$ for every p. Lemma 84 shows that under the retry invariant every σ_k is then well-formed, so no run of the sequence is refused.

3 Propagation

This section transcribes SPEC §9.2, the three-phase procedure that assigns each package a propagated channel, resolves every package's channel, and then assigns propagated bumps. The phase order is forced: phase 3 reads what phase 2 produces, and SPEC §9.2 forbids fusing them.

3.1 The procedure, as written

The pseudo-code is reproduced from SPEC §9.2 unchanged, so that a reader can diff it against `SPEC.md`. The definitions that follow are its clauses, one by one.

```
propagate(units, graph, held, W):
    edges = graph.restrictedTo(config.propagation.kinds)          # ğ8.4

    # ---------- shared, computed once ----------
    # sourcePackages is a pure function of the unit (ğ13.4a) and both passes read it.
    # Computing it inside each pass evaluates it twice for every unit carrying both axes.
    src = { u: sourcePackages(u) for u in units }                 # ğ13.4a

    # ---------- phase 1: channel axis ----------
    chan = {}                                   # package -> (channel, commit, unitIndex)
    for u in units:
        if u.channelDepth == 0:                     continue      # ğ8.3a: default
        if u.propagateChannel == none:              continue
        sources = src[u]
        if sources is empty:                        continue
        c      = commitOf(u)                                      # loop-invariant
        pscope = resolve(u.propagateChannelScope)                 # ğ8.5a: resolved once,
        origin = UNCOMPUTED                                      # memoized on first admitted target
        for (d, level) in reach(edges, sources, u.channelDepth):  #   never per target
            if d not in pscope:                           continue
            if c not in Wfresh(d):                             continue # admission, ğ13.4a
            if cancelledFor(c, d):                        continue # ğ13.5a
            if u.propagateChannel == 'inherit' and origin == UNCOMPUTED:
                origin = originChannel(u)
            v = propagatedChannelFor(u, d, origin)                # ğ9.3; origin computed once
            if v is NONE:                                 continue
            chan[d] = newerOf(chan[d], (v, u))                    # W160 on conflict

    # ---------- phase 2: channel resolution ----------
    channel = resolveChannels(chan, units)                        # ğ13.8

    # ---------- phase 3: bump axis ----------
    prop = {}                                   # package -> bump
    prov = {}                                   # package -> set of origin packages
    for u in units where bumpOf(u) != none:
        b = (u.propagate == 'inherit') ? bumpOf(u) : u.propagate
        if b == none or u.depth == 0:                continue
        sources = src[u]
        if sources is empty:                        continue
        c       = commitOf(u)
        pscope  = resolve(u.propagateScope)                       # ğ8.5: once per unit
        # ğ9.3a resolvability depends only on the unit's sources, so it is decided here,
        # once, and reduces to membership in the distinct source-channel set per target.
        srcChan   = { channel[P] : P in sources }                 # ğ9.3a
        anyStable = 'stable' in srcChan
        for (d, level) in reach(edges, sources, u.depth):
            if d not in pscope:                           continue
            owed = { P in sources : not delivered(P, c, d) }    # admission, ğ13.4a
            if owed is empty:                             continue
            if cancelledFor(c, d):                        continue
            if not (anyStable or channel[d] in srcChan):  continue # ğ9.3a, W208
            prop[d] = max(prop[d], b)
            prov[d] = prov[d] | owed
    return prop, prov, chan
```

The two axes admit differently, and the difference is the subject of this report's revision. Phase 1 tests `c not in Wfresh(d)`: a dependent is reached on the channel axis if and only if it has not itself released past the commit carrying the unit. Phase 3 tests `owed is empty`: a dependent is reached on the bump axis if and only if some source of the unit has not yet *delivered* that commit to it (Definition 10). The two coincide except for a dependent that released past the commit before its source did (Lemma 20), and `prov[d]` records exactly the

owed sources.

3.2 The traversal

Definition 28 (reach, SPEC §9.2)**.** For $S \subseteq \mathcal{P}$ and $\ell \in \mathbb{N} \cup \{\infty\}$,

$$R(S, \ell) \;=\; \{\, d \in \mathcal{P} \setminus S \;:\; \delta(S, d) \le \ell \,\},$$

and each $d \in R(S, \ell)$ carries the level $\delta(S, d)$.

```
reach(edges, sources, depth):
    frontier = sources
    seen     = set(sources)                    # a unit never reaches its own packages
    out      = {}                              # package -> level
    for level in 1 .. depth:                   # 'all' => until frontier is empty
        next = {}
        for p in frontier:
            for d in edges.dependentsOf(p):
                if d in seen: continue
                seen.add(d); next.add(d)
        if next is empty: break
        for d in next: out[d] = level
        frontier = next
    return out
```

Lemma 29 (the traversal computes Definition 28)**.** *On termination,* `out` *has domain* $R(S, \ell)$ *and* `out`$[d] = \delta(S, d)$.

Proof. Standard breadth-first search from the set S, seeded at level 0. By induction on `level`: `frontier` at the start of iteration n is exactly $\{d : \delta(S, d) = n - 1\} \setminus S$ for $n > 1$ and S for $n = 1$, because `seen` contains precisely the packages at distance $\le n - 1$ (together with S at level zero), and a package first enqueued at iteration n has a chain of length n from S and none shorter. Membership of `seen` is tested before insertion and never cleared, so each package is assigned at most one level, its least. The loop runs for ℓ iterations or until the frontier is empty. Seeding `seen` $= S$ removes S from the output, which is Definition 28's $\mathcal{P} \setminus S$. □

Three properties recorded by SPEC §9.2 are visible in this statement and are used repeatedly below. Depth is shortest-path, so a package reachable at both depth 1 and depth 3 is included by `+1`. Depth is measured from the originating source set as a whole and never recomputed from an intermediate package, so a unit written `+1` reaches exactly the direct consumers of its own packages in every run, whatever released in between. And a package reached by several paths is visited once, so a diamond does not double-count.

Example 30 (depth, on Figure 1)**.** Take $S = \{$`core`$\}$, the source set of a unit scoped `(core)`:

Written	$R(S, \ell)$	Note
`feat(core)`	$\emptyset$	$\ell = 0$, the default
`feat(core)^`	{`cli`, `ui`, `api`}	direct consumers
`feat(core)+2`	$R(S, 1) \;\cup$ {`docs-site`, `@acme/theme`}	two edges out
`feat(core)^^`	the same set	$\ell = \infty$; the graph is only two deep

Now take the two-package source set $S' = \{\texttt{core}, \texttt{ui}\}$, as a unit scoped `(core,ui)` would have. Then $R(S', 1) = \{\texttt{cli}, \texttt{api}, \texttt{docs-site}, \texttt{@acme/theme}\}$: `ui` is *absent* although it is a dependent of `core` at distance 1, because Definition 28 removes the whole of S'. The unit bumps `ui` directly, and a package is never both a direct and a propagated target of the same unit. Proposition F122 is about exactly this clause.

Remark 31 (self-exclusion is per unit). $R(S, \ell)$ excludes all of S, not merely the source a given path started from. This is what stops a unit from propagating to a package it already bumps directly. It is also the clause that makes the bucketed traversal of SPEC §13.11 unsound if transcribed naively; see Proposition F122.

3.3 Phase 1: the channel axis

Definition 32 (proposed channel, SPEC §9.3)**.** For a unit u and package d, let $cur = \mathrm{ch}(\mathrm{base}_\sigma(d))$ (stable if d has no baseline). Then $\mathrm{pcf}_\sigma(u, d) \in \mathcal{C}h \cup \{\bot\}$ is:

- if $\mathrm{pc}(u)$ is a transition $\langle f \to t \rangle$: $\bot$ if $f = t$ (`W207`); $\bot$ if cur does not match f, where $f = *$ matches any $cur \neq$ stable and otherwise f matches only $cur = f$; else $target = t$;
- otherwise $target$ is $\mathrm{pc}(u)$, or the origin channel of u where that is `inherit`; and if $target =$ stable while $cur \neq$ stable, the result is $\bot$ (`W200`: a propagated channel never graduates implicitly);
- in either case, if $target = cur$ the result is $\bot$ (`W199`); otherwise it is $target$.

The origin channel of u is the channel u's own `Channel` directive assigns to its sources if it carries one, and otherwise $\mathrm{ch}(\mathrm{base}_\sigma(p))$ for its sources p, taking the value of the byte-wise least name if they disagree (`W160`).

The final clause complements the target $W^\circ(a)$dmission gate: it compares $target$ against the package's *baseline* channel rather than against anything computed in the current run. This no-op check is the other premise of **G7**, and Theorem 89 is its consequence.

Definition 33 (phase 1 output, SPEC §9.2)**.** $\mathrm{prop}\chi_\sigma(d)$ is the value $\mathrm{pcf}_\sigma(u, d)$ of the $\prec$-greatest unit u (newest commit first, then last unit within a commit, SPEC §11.6) satisfying: $\mathrm{pcd}(u) \neq 0$; $\mathrm{pc}(u) \neq$ `none`; $\mathrm{src}_\sigma(u) \neq \emptyset$; $d \in R(\mathrm{src}_\sigma(u), \mathrm{pcd}(u))$; $d \in \mathrm{pcs}(u)$; $c_u \in W^\circ_\sigma(d)$; $\neg\mathrm{canc}(c_u, d)$; and $\mathrm{pcf}_\sigma(u, d) \neq \bot$. It is undefined if no unit qualifies.

The channel axis keeps $c_u \in W^\circ_\sigma(d)$ as its admission because a channel is carried by the units and needs no release of the source (SPEC §13.4a, **G7**); it has no notion of delivery.

3.4 Phase 2: channel resolution

Definition 34 (channel resolution, SPEC §13.8)**.** For every $p \in \mathcal{P}$, whether or not it will be released, let $base = \mathrm{ch}(\mathrm{base}_\sigma(p))$ and let $D_\sigma(p)$ be the value proposed by the $\prec$-greatest unit u with $p \in \mathrm{scope}(u)$, $c_u \in W^\circ_\sigma(p)$, $\neg\mathrm{canc}(c_u, p)$, that carries a `Channel` directive proposing something for p (a transition matching $base$, or a value differing from $base$; a directive proposing nothing is not a competitor and does not count towards `W186`). Then

$$\mathrm{chan}_\sigma(p) \;=\; \begin{cases} D_\sigma(p) & \text{if defined,} \\ \mathrm{prop}\chi_\sigma(p) & \text{else if defined,} \\ base & \text{otherwise.} \end{cases}$$

A direct directive beats every propagated one regardless of age; among direct directives, and separately among propagated ones, the newest commit wins and then the last unit within it. A package named by nothing keeps its baseline channel, which is why an established train needs no directives to stay together.

3.5 Phase 3: the bump axis

Definition 35 (resolvability, SPEC §9.3a)**.** $\mathrm{sch}_\sigma(u) = \{\mathrm{chan}_\sigma(p) : p \in \mathrm{src}_\sigma(u)\}$ and

$$\mathrm{rsv}_\sigma(u,d) \;\equiv\; \mathsf{stable} \in \mathrm{sch}_\sigma(u) \;\vee\; \mathrm{chan}_\sigma(d) \in \mathrm{sch}_\sigma(u).$$

A bump is a claim that the dependent has something new to pick up, and across a channel boundary the claim is false: a stable consumer resolves a beta-releasing dependency by its stable range exactly as before, so a republished consumer would contain byte-for-byte what its predecessor contained. Suppression is reported per suppressed pair as `W208`. Note that `srcChan` in the pseudo-code is computed over the whole source set, not over the owed sources; Section 8 returns to this.

Example 36 (a caret that reaches and does not bump: SPEC vector 95)**.** The unit `feat(core)^%beta: x` on Figure 1. The `%` puts `core` on `beta`; the `^` reaches $\{\texttt{cli}, \texttt{ui}, \texttt{api}\}$. Phase 2 leaves the three consumers where their baselines put them, on stable, so

$$\mathrm{sch}_\sigma(u) = \{\texttt{beta}\}, \qquad \mathsf{stable} \notin \mathrm{sch}_\sigma(u), \qquad \mathrm{chan}_\sigma(\texttt{cli}) = \mathsf{stable} \notin \mathrm{sch}_\sigma(u),$$

and Item (A6) fails for every one of them: three `W208` diagnostics, and $\Pi(\sigma) = \{\texttt{core}\}$ alone. The caret was honoured and reached its targets; what it would have delivered is not resolvable by them. Writing `feat(core)^%beta++1` instead moves the same three onto `beta` in phase 1, so $\mathrm{chan}_\sigma(\texttt{cli}) = \texttt{beta} \in \mathrm{sch}_\sigma(u)$ and all four packages release, which is the form that keeps a train installable as a set.

Definition 37 (admission, SPEC §9.2, SPEC §13.4a)**.** For $u \in \mathcal{U}$ and $d \in \mathcal{P}$, $\mathrm{adm}_\sigma(u,d)$ holds iff all of:

(A1) $\mathrm{bump}(u) \neq \mathsf{none}$, $\mathrm{pb}(u) \neq \mathsf{none}$, $\mathrm{pd}(u) \neq 0$;
(A2) $\mathrm{src}_\sigma(u) \neq \emptyset$ and $d \in R(\mathrm{src}_\sigma(u), \mathrm{pd}(u))$;
(A3) $d \in \mathrm{ps}(u)$;
(A4) $\mathrm{owed}_\sigma(u,d) \neq \emptyset$;
(A5) $\neg\mathrm{canc}(c_u, d)$;
(A6) $\mathrm{rsv}_\sigma(u,d)$.

Definition 38 (propagated and direct bumps, SPEC §13.6, SPEC §13.7)**.**

$$\mathrm{prp}_\sigma(d) = \bigsqcup\{\, \mathrm{pb}(u) \;:\; u \in \mathcal{U},\ \mathrm{adm}_\sigma(u,d) \,\},$$
$$\mathrm{dir}_\sigma(p) = \bigsqcup\{\, \mathrm{bump}(u) \;:\; p \in \mathrm{src}_\sigma(u) \cap \mathrm{scope}(u),\ c_u \in W^\circ_\sigma(p) \,\},$$

both empty joins being stable. Then $\mathrm{eff}_\sigma(p) = \mathrm{dir}_\sigma(p) \sqcup \mathrm{prp}_\sigma(p)$.

Item (A4) is the load-bearing clause of the entire design, and it is worth being explicit about what it is and is not. It tests the *target's* position against its *sources'* releases: d is admitted while some source has not delivered c_u to it. It does not test the source's window, and it does not test the target's window either. SPEC §13.7a shows what each of the other readings costs. With retention governed by the source's window, a dependent whose publish failed while

its provider's succeeded is silently orphaned, permanently one patch behind a dependency it declares, with no diagnostic, because from the engine's point of view nothing is pending. With admission governed by the target's window, the rule of CCME 2.0.0, the same orphan is reached from the other side: a dependent that proceeded past the commit on a cause of its own while its provider failed is owed nothing once the provider publishes, and keeps the range it reconciled against the old version. By Lemma 20 the delivery test agrees with the window test on every target that has not released past c_u; it differs only on a target that has, where it asks the finer question. Every guarantee in Section 6 rests on Item (A4) being this test, and Theorem 74 is where that is used.

Example 39 (the orphan, from both sides: SPEC vectors 66 and 80d)**.** Both shapes use the history of Example 14, $c_0 \to c_1 \to c_2 = h$, with the tags of σ_0 at c_0 and the unit $u =$ `feat(core)^: streaming reader` at $C = c_2$. So $\mathrm{scope}(u) = \{\texttt{core}\}$, $\mathrm{bump}(u) =$ minor, $\mathrm{pb}(u) =$ patch (the default of Definition 16) and $\mathrm{pd}(u) = 1$.

Provider fails to deliver; the consumer waits (vector 66). Run 1 evaluates Definition 37 with σ_0. All of `cli`, `ui`, `api` lie in $R(\{\texttt{core}\}, 1)$, none is cancelled, all are on stable as is `core`, and $C \in W^\circ_{\sigma_0}(d)$ for each, so by Lemma 20 $\mathrm{owed}_{\sigma_0}(u, d) = \{\texttt{core}\}$, all three are admitted and

$$\Pi(\sigma_0) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}, \qquad \nu_{\sigma_0}(\texttt{core}) = 1.5.0, \quad \nu_{\sigma_0}(\texttt{cli}) = 2.0.1.$$

Suppose `core`, `ui` and `api` publish and `cli` fails, so

$$\sigma_1 = \sigma_0 \cup \{\texttt{core@1.5.0},\ \texttt{ui@0.9.2},\ \texttt{api@1.2.1}\} \quad \text{at } C.$$

Run 2 is where the readings of Item (A4) separate. Under delivery, the normative reading, $C \not\preceq \mathrm{basec}_{\sigma_1}(\texttt{cli}) = c_0$, so $\neg\mathrm{dlv}_{\sigma_1}(\texttt{core}, C, \texttt{cli})$ and $\mathrm{owed}_{\sigma_1}(u, \texttt{cli}) = \{\texttt{core}\}$: `cli` is admitted. Under the target's window, the reading of CCME 2.0.0, $C \in W^\circ_{\sigma_1}(\texttt{cli}) = \{c_1, C\}$: `cli` is admitted. Under the source's window, the trap, $C \notin W_{\sigma_1}(\texttt{core}) = \emptyset$: u reaches nobody. The first two agree, as Lemma 20 says they must for a `cli` that has not released past C. For `ui`, which has, $\mathrm{dlv}_{\sigma_1}(\texttt{core}, C, \texttt{ui})$ holds with $t = C \preceq C = \mathrm{basec}_{\sigma_1}(\texttt{ui})$: nothing is owed, and `ui` is out. Under the normative rule $\Pi(\sigma_1) = \{\texttt{cli}\}$ at the version run 1 planned, marked `W193`, and $\Pi(\sigma_2) = \emptyset$. Under the trap $\Pi(\sigma_1) = \emptyset$ and `cli` is never released, on this run or any later one, with no diagnostic anywhere.

Consumer gets ahead of its provider (vector 80d). Let C carry a second unit $u' =$ `feat(cli): y` beside u. Then $\mathrm{dir}_{\sigma_0}(\texttt{cli}) =$ minor, and run 1 plans $\Pi(\sigma_0) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}$ with $\nu_{\sigma_0}(\texttt{cli}) = 2.1.0$. Suppose `core` fails. `ui` and `api` have no cause of their own relative to $X = \{\texttt{core}\}$ and are blocked (`W194`); `cli` has one, its direct minor, and *proceeds* (Definition 23): it is published at 2.1.0 with its manifest reconciled to `core`'s published baseline 1.4.2, and tagged at C. So

$$\sigma_1 = \sigma_0 \cup \{\texttt{cli@2.1.0 at } C\}, \qquad W^\circ_{\sigma_1}(\texttt{cli}) = \emptyset.$$

Run 2. `core` is planned by its direct minor. For `cli`, the window reading says $C \notin W^\circ_{\sigma_1}(\texttt{cli})$ and admits nothing: `cli` keeps the range it reconciled against 1.4.2, is never planned again, and is one minor behind a dependency it declares, silently. The delivery reading asks whether `core` has delivered C to `cli`: $T_{\sigma_1}(\texttt{core}) = \{c_0\}$ and $C \not\preceq c_0$, so $\mathrm{owed}_{\sigma_1}(u, \texttt{cli}) = \{\texttt{core}\}$, `cli` is admitted with $\mathrm{prp}_{\sigma_1}(\texttt{cli}) =$ patch, and

$$\Pi(\sigma_1) = \{\texttt{core}, \texttt{cli}, \texttt{ui}, \texttt{api}\}, \qquad \nu_{\sigma_1}(\texttt{cli}) = 2.1.1,$$

ordered after `core` and blocked if `core` fails again. σ_1 is well-formed (Definition 24): `core` would be released at $C = \mathrm{basec}_{\sigma_1}(\texttt{cli})$ and still owes `cli`, and `cli` is in the plan after it. Once both are tagged, $\mathrm{dlv}_{\sigma_2}(\texttt{core}, C, \texttt{cli})$ holds with $t = C$, nothing is owed, and $\Pi(\sigma_2) = \emptyset$. The plan of run 2 equals the plan of run 1 although run 1 published a package; that is the observation Theorem 85 has to accommodate.

Example 40 (two tags on one commit: `E201`, vector 80d continued)**.** Continue the second shape from σ_1. Suppose run 2 publishes `core`, `api` and `ui` and `cli` fails. Then $\sigma_2 = \sigma_1 \cup \{\texttt{core@1.5.0}, \texttt{api@1.2.1}, \texttt{ui@0.9.2}\}$ at C, and now $\mathrm{dlv}_{\sigma_2}(\texttt{core}, C, \texttt{cli})$ holds with $t = C \preceq C = \mathrm{basec}_{\sigma_2}(\texttt{cli})$: the ledger reads `cli` as served, although `cli@2.1.1` was never published and `cli@2.1.0` still names 1.4.2. $\mathrm{owed}_{\sigma_2}(u, \texttt{cli}) = \emptyset$, $\Pi(\sigma_2) = \emptyset$, and no later plan at any commit will compute the debt, because two tags on one commit have no order. This is Proposition 26: run 2 reports `E201` for the pair, and the remedy is an explicit `Release-As` on `cli` at a later commit. The other route to the same state is prevented rather than reported: a run 2 in which `cli` sat out, held or deselected, would release `core` at `cli`'s baseline commit without `cli` following, and SPEC §19.3 refuses it before any publication. Had `core` instead published at a later commit c_3 in a run `cli` sat out, C would stay in $W^{\mathrm{ow}}(\texttt{core}, \texttt{cli}) = \mathrm{reach}(c_3) \setminus \mathrm{reach}(c_0)$ (Definition 11) and the next run would plan `cli`'s catch-up.

Remark 41 (catch-up is not a mode)*.* There is no separate catch-up pass in this model, and none in the specification. Catch-up is what the algorithm does when the ordinary rule is evaluated against the right position (SPEC §13.7a). A "catch-up release" is a label for a release whose entire cause is a propagation from a package not itself in the run's plan (`W193`); the engine does nothing different to produce it, and vector 80d's second run is not one, since `core` is in that plan. Catch-up does not widen depth, resurrect a cancelled contribution, release a held package, change a planned version, or propagate onward from the catching-up package; each of which is a property of Definition 37 rather than a rule about catch-up.

4 The publish protocol

The guarantees are joint properties of the computation and the protocol that writes the state the next computation reads. SPEC §19 says so explicitly: they do not survive an implementation that gets this section wrong. This section transcribes the obligations the proofs use, in the order a run meets them: the comparison of release records under the lock, tagging, publication order, build readiness, partial failure, reconciliation, and manifest writes. The durations these obligations lead to, and their distributed and multi-repository forms, are the subject of Section 5.

4.1 Records under the lock

Definition 8 takes the state σ to be the set of release tags reachable from h. SPEC §13.2 says whose tags: the authoritative store's, not the checkout's.

Definition 42 (records under the lock, SPEC §13.2)**.** A write-capable run plans from the release records of the authoritative store it records to, not from whatever its checkout fetched. After it holds every release lock it needs and before it fixes the input of SPEC §13.2, it compares, for each participating repository, the release records of the store with the records it is about to plan from. A stored record whose commit is reachable from the planned head and which the planning input lacks is incomplete history, `E196`. A stored record that names another commit than the planning input's record of the same package and version is `E191`. Neither difference is repaired by planning the package as unreleased, and the records are not refreshed after the comparison: a run that fetches does so before it and plans from what it then holds. Only what SPEC §12.1 parses as a release tag of a workspace package is a record here; a ref the implementation moves by design, its lock, and a name that merely resembles a tag are not. A store whose records cannot be read is `E196`. Read-only planning takes no lock and makes

no comparison; its result describes the checkout. The comparison reads one record inventory per repository, $O(T)$ each, and asks one ancestry question per record the input lacks, which is none in the ordinary case.

The lock alone serialises runs and isolates nothing: two runs that never overlap still plan the same version when the second one's checkout predates the first one's records. The comparison is what the lock makes sufficient, because no other coordinated run can add a record between the comparison and this run's own. Complete history makes reachability decidable locally, since a commit the checkout does not hold cannot be reachable from its head. A run under an unsafe lock bypass (`W331`) still compares; nothing then holds other runs off after the comparison, so the exposure is narrowed and not closed. Under SPEC §27 the inventory is read per owning repository and cross-repository consumer positions are reconstructed as Section F5.3 describes; commit identifiers of different repositories are never compared as one ancestry relation.

4.2 Tag after, immediately, always, and once

Definition 43 (tagging, SPEC §19.1)**.** For each released package: the tag is created at h; *after* that package's publish succeeds, never before; *immediately* after it, not batched to the end of the run; for every released package whatever its publish target, including targets that upload no artefact at all; and *once*. A release tag is created, never replaced. Writing it to the authoritative store fails when the store already holds that tag name, unless the stored tag names the same commit for the same package and version, which is the retry of an uncertain write (Definition 55). A forced ref update is never used for a release tag, and a tag is not moved or deleted once pushed. A ref the implementation moves by design is not a release record and is outside this rule. For a target of `none` there is no publish step: the tag is written once the package's manifest write succeeds.

Each clause excludes a specific failure. Tagging before publishing makes the version unrecoverable, because Definition 9 will treat it as released. Batching tags to the end of a run makes a mid-run failure republish everything already published, which most registries reject outright. Tagging only what uploads an artefact is the subject of Proposition 88. The create-only clause is what makes Definition 8 monotone in fact and not only by convention: a record, once written, is never overwritten by a later writer, which is also what SPEC §28.6 relies on when an orchestrator records a publication it authorised before losing its lock (Definition F70).

4.3 Order

Definition 44 (publish order, SPEC §19.2)**.** Let G be the workspace graph restricted to `publish.orderKinds` over *all* packages. Publication proceeds in a topological order of G, ties broken by byte-wise least name, filtered to the plan at the end.

Proposition 45 (the order exists, is total, and is deterministic)**.** *Definition 44 is well defined and yields the same sequence on every machine and in every conforming implementation.*

Proof. G is acyclic by Definition 3, so a topological order exists. Ties are broken by a total order on names (byte-wise comparison of distinct names), so the order is total and independent of any container's iteration order. The filter is applied to a sequence, preserving it. □

Remark 46 (order over the whole graph, then filter). Inducing the graph on the planned packages *first* is a defect, not an optimisation. With $A \to B \to C$ where B has no bump this

run (the common case in any incremental release), the induced subgraph on $\{A, C\}$ has no edge at all, so A may publish against a C that has not published yet. The constraint is real even though B is not released, because A still resolves C through B at install time. This is why the model orders over $\mathcal{P}$ and filters last, and why an implementation detecting a violation must raise `E197`.

Order matters for a reason no later run can repair. A dependent published before its dependency carries a range reconciled against the dependency's old version, and both land at the same commit, so no ancestry test can tell afterwards that anything is wrong (SPEC §13.7b). The inconsistency is undetectable after the fact, which is why it is prevented rather than diagnosed. `E197` is scoped to Definition 44 and not to propagation: every workspace dependency edge between two planned packages constrains the order, whether or not either package's bump arrived by propagation, because a dependent planned for its own unrelated `feat` still resolves its dependency at install time (SPEC §22, vector 80a). An `E197` that fires only for propagated pairs is under-reporting.

4.4 Build readiness

Definition 44 orders publication. An engine that also runs the packages' builds has a second question for every edge between two packages that build in this run: what the consumer's build reads of the provider.

Definition 47 (build readiness relation, SPEC §19.2a)**.** For every workspace dependency edge $C \to P$ between two packages that build in this run, the edge carries a declared *build readiness relation* with one of three values, ordered `none` < `build` < `publish`:

- `publish`: C's build reads P as an installer resolves it, the published artefact and its record, and starts after P is published and recorded (Definition 43);
- `build`: C's build reads what P's build leaves in the workspace, and starts after P's build has succeeded;
- `none`: C's build reads nothing that P's build or publication produces in this run, and waits for nothing of P.

C's build includes every preparation step of C that can read the provider, lockfile regeneration in particular; manifest reconciliation (Definition 56) reads planned versions only and waits for nothing. Each precondition implies the one before it, because a package builds before it publishes. Under all three values P publishes before C does: Definition 44 is unchanged, is not a fourth value, and cannot be declared away. The relation is *declared, never inferred*: an engine cannot check that a build reads nothing, so it neither infers `none` nor makes it a default. It is *policy, not plan*: it never changes which packages release or at which versions, is no part of the input of SPEC §13 or of a plan digest, and two runs of one plan under different relations record the same releases when every task succeeds; under SPEC §28 it travels with the run's resolved configuration and a node's own settings cannot change it. It is *ordered over the whole graph*: for two packages C and P that build in this run, a dependency path from C to P with no `none` edge on it places P's build before C's, whether or not the packages between them are in the plan, and a `none` edge ends the constraint of every path through it.

The reference implementation reads the relation at the consumer's own edge only, and carries build and publish *order* through a non-building intermediate by a memoised reach (`release/stagerelation.go`), never the intermediate's relation value; that is the reading taken here for a `publish` edge behind a package that does not build, and the question is flagged for the specification (Section F10).

`none` therefore means build in parallel and publish in order, which is the relation of a deployment order: infrastructure before the application that runs on it, a schema before the service that reads it. Vector 80c of SPEC §22 is the whole-graph clause: with `app` → `ui` → `core` where `app` and `core` build and `ui` does not, `core`'s build precedes `app`'s when both edges are `build`, and no build constraint exists when either edge is `none`, while `core` publishes before `app` in every row. Adding build edges only between a package and its direct providers in the plan loses the first row, exactly as inducing the publish graph on the plan loses vector 80a.

Definition 48 (task graph, SPEC §19.2a)**.** The *task graph* of a run has a node build(X) for every package X that builds and a node publish(X) for every $X \in \Pi(\sigma)$, and the edges (arrows denote happens-before): build(X) → publish(X); publish(P) → publish(C) whenever P precedes C in Definition 44 along a dependency path; build(P) → build(C) for every dependency path from C to P with no `none` edge, both building; and publish(P) → build(C) for every edge $C \to P$ declared `publish`.

Proposition 49 (no new cycle, SPEC §19.2a)**.** *For every choice of relations the task graph is acyclic. Weakening a relation only removes edges.* ***`E197`*** *cannot arise from a relation.*

Proof. Let r number the packages in a topological order of the workspace graph with providers first, so that $r(P) < r(C)$ whenever C depends on P over a dependency path. Give each task the key $(r(X), s)$ with $s = 0$ for a build and $s = 1$ for a publication, ordered lexicographically. Every edge of Definition 48 strictly increases the key: build(X) → publish(X) raises s at fixed r, and the three remaining forms raise r, since each runs along a dependency path from a consumer C to a provider P with $r(P) < r(C)$. A cycle would return to its starting key, which a strictly increasing key cannot do. The only edges a relation contributes are of the third and fourth forms; `publish` contributes both, `build` the third, `none` neither, so a weaker value contributes a subset. The publication order of Definition 44 reads no relation, so a violation of it (`E197`) cannot be caused by one. □

Failure is the business of Definition 50. Under `build` and `none`, C may be prepared and built against P's planned version before the outcome of P's publication is known. That is sound because Definition 50 then either blocks C or, where C has a cause of its own, reconciles its manifest to P's published version before C publishes and leaves P's contribution owed (Definition 19): a manifest naming a version that was never published is never published itself. A build that embedded the planned version is rebuilt, or C is blocked; a build finished for a blocked consumer confers nothing, is not published, discharges no obligation, and the next run builds again or reuses it under the identity rules of SPEC §28.5. Under SPEC §28 a `build` edge is the local-output dependency and a `publish` edge the registry-availability edge of Definition F69; a `none` edge carries no output into the consumer's build task, and the requirement that task inputs include every build dependency is met because the declaration says there is none.

4.5 Partial failure

Definition 50 (own causes, blocking, and proceeding, SPEC §19.3)**.** Let $F \subseteq \Pi(\sigma)$ be the packages whose publish failed. A package $d \in \Pi(\sigma)$ that transitively depends over `publish.blockingKinds`, through the *full* workspace graph, on a member of F or on a blocked package is *blocked* (`W194`) when it has no cause of its own, and *proceeds* otherwise. A cause of its own is a fresh direct bump, a channel change, or a contribution from a provider that is neither failed nor blocked, including one that published in an earlier run. A blocked package is planned but not attempted in this run. A proceeding package publishes at its planned version

with its manifests reconciled to what its providers have published (Definition 56); what the failed provider owed it stays owed (Definition 19) and arrives as a catch-up (`W193`) when that provider publishes. Packages that depend on no failed or blocked package continue to publish: an unrelated subtree is not punished for another's failure. The run reports what published, what failed and what was blocked, and exits non-zero.

The closure traverses packages that are not in the plan: a package with no bump this run is still a path from a dependent to a failed dependency (vector 80a). Ordering and blocking use different edge-kind sets, and the asymmetry is deliberate: `orderKinds` includes optional dependencies, because an optional dependency should still be available before its dependent, while `blockingKinds` excludes them, because a missing optional dependency does not break an install and a missing required one does. The pseudo-code of SPEC §19.3 is transcribed verbatim (the section sign is mapped to `§` for the typewriter font and nothing else is changed):

```
run(plan):
    published, failed, blocked = {}, {}, {}
    for P in publishOrder(plan, graph):            # a flat sequence, §19.2
        bad = transitiveBlockingDeps(P, graph) & (failed | blocked)   # full graph, §19.3
        if bad and ownCauses(P, plan, bad) is empty:
            blocked[P] = W194; continue
        reconcileRanges(P, published)              # §9.4, §19.5: providers as published by now
        try:
            if publishTarget(P) != none:           # §13.10a
                publishTo(publishTarget(P), P, plan[P].version)   # or adopt, §19.4
            tag(P, plan[P].version, HEAD)          # §19.1: always, immediately
            published[P] = plan[P].version
        except PublishError as e:
            failed[P] = e
            if config.publish.onFailure == 'abort': return report(...)
    return report(published, failed, blocked)
```

`publishOrder` returns a flat, totally ordered sequence, not levels, so `run` iterates it directly. Publishing strictly one at a time is the normative reading; an implementation may publish concurrently only if it preserves the same happens-before relation and reports in the sequence order regardless (SPEC §17.2). On re-run the plan is recomputed from tags: packages already tagged fall out by SPEC §13.6, packages that failed or were blocked remain candidates by Definition 19, and a dependent that proceeded returns as a catch-up once the failed provider publishes.

Remark 51 (the blocked set of Definition 25). Definition 25 takes blocking in exactly this sense: its blocked set is the closure of the failed set restricted to packages without a cause of their own relative to what failed or was blocked before them in the publish order (Definition 23), and a proceeding dependent belongs to its published set at its planned version. The proofs of Section 6 that quantify over $F \cup B$ use only that a package in $F \cup B$ gained no tag. What a proceeding dependent is still owed is the subject of Definition 19 and of the delivery rule it transcribes, not of the blocking closure.

Definition 52 (the owed-commit rule, SPEC §19.3)**.** A package P is not released at the baseline commit of a consumer D it still owes (Definition 19) unless D is in the plan and is published after P in this run. Such a run is refused with `E201` before any publication, naming the pair; the remedy is to release the consumer in the same run, or the provider at a later commit, where the debt stays visible in the owed window of SPEC §13.3. If D is published after P and fails, the same state arises, and `E201` reports it with its remedy, an explicit `Release-As: <version>` on the consumer (SPEC §8.6), because no later plan can compute the debt.

Definition 24 states the rule as a well-formedness condition on the state a run starts from, and Definition 25 refuses a run from a state that violates it. Two releases on one commit cannot be ordered afterwards (SPEC §13.7b). A consumer that sat the run out, held, deselected or excluded by any other means, would then read as served: the delivery test of SPEC §13.4a finds a release of P on a commit that D's baseline reaches, namely D's own baseline commit, and nothing is left to detect that D came first. The rule prevents that state rather than diagnosing it.

Example 53 (an infrastructure leg)**.** Nothing in Definitions 25 and 43 requires a publish leg to be a registry upload. A `terraform apply` may be such a leg: its build stage imports the explicitly configured live resources and writes a plan, and its publish stage applies that plan before the release tag is written. The tag records that this version of the configuration completed successfully. It is not Terraform resource state, does not prove that every live resource was imported, and does not make applying an arbitrary configuration externally idempotent. The edge from the application to the infrastructure package is the case Definition 47 names: the application's build reads nothing the apply produces, so the edge is declared `none`, the two builds overlap, and only the application's publication waits for the infrastructure publish. Example 64 gives the durations.

Remark 54 (recorded-success abstraction)*.* Definition 25 models confirmed successful legs by their tags. It proves properties of that release ledger, not convergence of an external registry or infrastructure state. A publish may take effect before its tag is recorded; then the repository alone is insufficient and Definition 55 or a publish-target-specific idempotence contract is required. Terraform likewise needs its own state or an explicit, complete import boundary before a plan can safely describe existing resources.

4.6 Reconciliation and manifest writes

Definition 55 (reconciliation, SPEC §19.4)**.** When a publish fails because the version already exists, the implementation verifies by registry digest or provenance attestation that the published artefact is the one this run would have published; if it matches, the publish counts as successful and the tag is written (`W196`); if it does not match, or identity cannot be established, the run stops with `E198`.

This is the one place where the model's "a publish either succeeds or fails" abstraction meets a third outcome, that it already happened, and Definition 55 maps it back onto success. Verification is what makes that safe: without it, adopting a version published by something other than this run would enrol a foreign artefact into the release lineage, which is a supply-chain vulnerability rather than a convenience.

Definition 56 (manifest writes, SPEC §19.5)**.** Before a released package publishes, its manifest is updated so that it declares no range excluding the version each workspace dependency has *published* by the time the package is reconciled: the dependency's planned version when it is in this run's plan and has published, and its current baseline otherwise (`W197` when that baseline came from an earlier run). A dependency that is in the plan but failed or was blocked has published nothing new; a dependent that proceeds past it (Definition 50) is reconciled to the baseline, and `W193` follows when the dependency publishes. Because the graph is acyclic and Definition 44 publishes every dependency before its dependents, the planned and published versions coincide at the moment each package is reconciled whenever the dependency published; reconciliation may therefore be computed once up front from the plan and corrected, before the dependent's publish, only for the providers that did not publish. A manifest naming a version that was never published is never published, whatever was computed up front. The

writes are part of the publish step, not the plan, and are not committed back in a way that itself parses as a bump-producing unit.

5 Execution: what readiness and placement cost

Section 4 states what must happen before what. This section states what that costs in wall time, where the tasks may run, and which history each window ranges over when the packages live in several repositories. The duration propositions are informative in the sense of SPEC §13.11: they bound durations of schedules that respect Definition 47, and no conforming implementation is required to attain them. The two optional profiles of SPEC §28 and SPEC §27 are transcribed as definitions, with the one invariance each promises, in Sections F5.2 and F5.3. Nothing here extends the theorems of Section 6, which are stated for a single history.

5.1 Build readiness durations

SPEC §19.2a promises a happens-before relation and no duration; SPEC §13.11 nevertheless says what each relation buys when nothing else limits the run. Throughout, package i has a build of duration b_i and a publication of duration p_i, both fixed; a task starts as soon as its predecessors in the task graph of Definition 48 have finished and a slot is free; and one relation is declared on every edge.

Proposition 57 (the longest path, SPEC §13.11)**.** *With unlimited build and publication slots a run lasts exactly the longest path L of its task graph, measured in task durations. A weaker relation never lengthens it.*

Proof. The task graph is acyclic (Proposition 49), so its tasks can be taken in a topological order. With unlimited slots a task starts at the greatest finish time of its predecessors, or at 0 if it has none, and by induction along that order its finish time is the longest path ending at it. The run ends at the greatest finish time, which is the longest path of the graph. A weaker relation contributes a subset of the edges (Proposition 49), so every path of its task graph is a path of the stronger one's, and the longest is no longer. □

Proposition 58 (a chain, SPEC §13.11)**.** *Let packages $1, \dots, n$ form a chain in which package $k+1$ consumes package k, under one relation throughout. With unlimited slots the run lasts*

$$\sum_{i=1}^{n}(b_i + p_i) \quad \textit{under } \texttt{publish}, \qquad \max_{1\le k\le n}\Big(\sum_{i=1}^{k} b_i + \sum_{i=k}^{n} p_i\Big) \quad \textit{under } \texttt{build},$$

$$\max_{1\le k\le n}\Big(b_k + \sum_{i=k}^{n} p_i\Big) \quad \textit{under } \texttt{none}.$$

Proof. Under `publish`, build$(k+1)$ waits for publish(k), so the $2n$ tasks form one path and the sum is its length. Under `build`, the builds form the path build$(1) \to \cdots \to$ build(n) and build(k) ends at $B_k = \sum_{i\le k} b_i$; the publication of k waits for build(k) and for publish$(k-1)$, so its finish E_k satisfies $E_k = \max(B_k, E_{k-1}) + p_k$ with $E_0 = 0$. By induction $E_k = \max_{j\le k}(B_j + \sum_{i=j}^{k} p_i)$: the case $k = 1$ is $B_1 + p_1$, and the step is $\max(B_k + p_k,\ \max_{j<k}(B_j + \sum_{i=j}^{k-1} p_i) + p_k)$. Under `none` every build starts at 0 and build(k) ends at b_k; the same recurrence with b_k in place of B_k gives the third form. In each case the run lasts E_n by Proposition 57. □

Corollary 59 (equal durations)**.** *With $b_i = b$ and $p_i = p$ for all i, the three forms are $n(b+p)$, $\max(nb+p,\ b+np)$ and $b+np$.* **`build`** *saves $(n-1)\min(b,p)$ over* **`publish`***, and* **`none`** *saves a further $(n-1)\max(b-p,0)$, so the second saving exists only where builds outlast publications.*

Proof. Under `build` the maximand $kb+(n-k+1)p$ is affine in k, so the maximum is at $k=1$ or $k=n$. Under `none` the maximand $b+(n-k+1)p$ is greatest at $k=1$. The differences follow by cases on $b \geq p$. □

Proposition 60 (fan-out and fan-in, SPEC §13.11)**.** *For one provider 0 and consumers $i \geq 1$, each consuming 0 and nothing else, the run lasts*

$$b_0 + p_0 + \max_i(b_i + p_i), \qquad b_0 + \max_i\big(\max(b_i, p_0) + p_i\big), \qquad \max_i\big(\max(b_i,\ b_0 + p_0) + p_i\big)$$

under **`publish`***,* **`build`** *and* **`none`** *respectively. For providers $i \geq 1$ of one consumer c, each consumed by c and nothing else, it lasts*

$$\max_i(b_i+p_i)+b_c+p_c, \qquad \max\big(\max_i b_i+b_c,\ \max_i(b_i+p_i)\big)+p_c, \qquad \max\big(b_c,\ \max_i(b_i+p_i)\big)+p_c.$$

Proof. Fan-out. Under `publish`, consumer i's build starts at b_0+p_0 and its publication follows its build, giving $b_0+p_0+b_i+p_i$; take the maximum. Under `build`, its build starts at b_0 and ends at b_0+b_i, and its publication waits for that and for publish(0) at b_0+p_0, giving $b_0+\max(b_i,p_0)+p_i$. Under `none`, its build ends at b_i and its publication waits for that and for b_0+p_0. Fan-in. Under `publish`, c's build starts when the last provider has published, at $\max_i(b_i+p_i)$. Under `build`, it starts at $\max_i b_i$ and ends b_c later, and c's publication waits for that and for every provider's publication. Under `none`, c's build ends at b_c and only its publication waits for the providers. Each is the longest path of Proposition 57. □

Proposition 61 (one publication lane, SPEC §13.11)**.** *Where one owner's publications are serialised (SPEC §27.2, SPEC §28.6) in the order $1,\dots,n$ of Definition 44, let e_k be the instant the build of package k ends. Under every relation the run lasts*

$$\max_{1\leq k\leq n}\Big(e_k + \sum_{i=k}^{n} p_i\Big),$$

which is at least $\sum_i p_i$ whatever the relation, and a weaker relation only lowers the e_k.

Proof. Publication k starts at $\max(e_k, E_{k-1})$, since it waits for its own build and for the lane, and the lane order agrees with Definition 44, so every provider of k has published by E_{k-1}. The recurrence is that of Proposition 58 with e_k in place of B_k, and the same induction gives the maximum. The term $k=1$ is $e_1+\sum_i p_i \geq \sum_i p_i$. For the last claim, add the lane edges publish$(k-1) \to$ publish(k) to the task graph; they follow the publish order, which is a topological order of the workspace graph, so the key of Proposition 49 still increases along every edge and the graph stays acyclic. With a fixed lane order every task starts at the greatest finish of its predecessors, and by induction along a topological order each finish time under a subset of the edges is at most the finish time under the full set; e_k is such a finish time. □

Proposition 62 (budgets, SPEC §13.11)**.** *Let there be m_b build slots and m_p publication slots, and write $W_b = \sum_i b_i$, $W_p = \sum_i p_i$ and L for the longest path of the task graph. No schedule finishes before* $\max(L,\ W_b/m_b,\ W_p/m_p)$*. A schedule that never leaves a slot of a stage idle while a task of that stage is ready finishes within $L + W_b/m_b + W_p/m_p$, which is within three times the best possible. Under budgets a weaker relation can lengthen such a schedule, but never the best one.*

Proof. The lower bound: a schedule contains every path, so it lasts at least L; it performs W_b units of build work on m_b slots, so at least W_b/m_b; and likewise W_p/m_p. The upper bound: let the schedule end at T and let t_1 be a task finishing at T. For $j \geq 1$, while t_j has predecessors, let t_{j+1} be one that finishes last among them; this gives a chain $t_q \to \cdots \to t_1$ whose total duration is at most L. Consider an instant of $[0, T)$ at which no task of the chain is running. It lies between the finish of some t_{j+1} and the start of t_j (or before the start of t_q), and at that instant t_j is ready, since its latest predecessor has finished, and has not started; by hypothesis every slot of t_j's stage is then busy. The instants at which all m_b build slots are busy total at most W_b/m_b, and those at which all m_p publication slots are busy at most W_p/m_p. Hence $T \leq L + W_b/m_b + W_p/m_p$, and each summand is at most the lower bound. For the last claim, a schedule feasible under a stronger relation is feasible under a weaker one, so the best weaker schedule is no longer; that a particular greedy schedule can be longer is Example 63. □

The bound is Graham's argument for list scheduling under precedence constraints [12, 13], applied to two stages with separate budgets; SPEC §13.11 does not claim it tight for this shape of graph.

Example 63 (Graham's anomaly, SPEC §13.11)**.** Take two build slots and one publication slot, packages $A = (2, 3)$ and $B = (1, 2)$ where B consumes A, and an unrelated $C = (8, 3)$, with ready tasks taken in the order A, B, C. Under `build`, B's build is not ready at 0: A builds in $[0, 2)$ and C in $[0, 8)$ on the two slots; B builds in $[2, 3)$; A publishes in $[2, 5)$, B in $[5, 7)$ once A has published, and C in $[8, 11)$ once its build ends. The run lasts 11. Under `none`, B's build is ready at 0 and takes the second slot: A builds in $[0, 2)$ and B in $[0, 1)$, so C builds only in $[1, 9)$; A publishes in $[2, 5)$, B in $[5, 7)$ and C in $[9, 12)$. The run lasts 12: the consumer's early build took the slot the long build would have had. A list that preferred C to B at 0 would finish at 11 under `none` as well, which is the point: the length depends on the list, not on the relation, and it is why SPEC §19.2a promises a happens-before relation and no duration.

Example 64 (an infrastructure chain)**.** Let the infrastructure leg of Example 53 be consumed by a backend consumed in turn by a frontend, with (b, p) in minutes of $(4, 6)$ for the infrastructure, $(9, 3)$ for the backend and $(12, 2)$ for the frontend. By Proposition 58, under `publish` the run lasts $10 + 12 + 14 = 36$ minutes; under `build` it lasts $\max(4 + 11,\ 13 + 5,\ 25 + 2) = 27$; under `none` it lasts $\max(4 + 11,\ 9 + 5,\ 12 + 2) = 15$. The publications still run in order in every case, and the frontend's publication still ends the run.

6 The eight guarantees

Throughout this section, h and the configuration are fixed and $\sigma_1 \leq \sigma_2 \leq \cdots$ is a run sequence (Definition 27). Four hypotheses are stated once and used by name.

(H1) stable inherited-channel state
every baseline channel read by `Propagate-Channel: inherit` remains unchanged through the sequence.

(H2) no suppressed source is released
for every unit u and every $p \in \text{scope}(u)$ whose contribution to u is suppressed at σ_1, whether by cancellation, by a hold (Definition 19), or by a correction (Remark 17), p is not released at h in the sequence.

(R) owed sources reach their target
for every u and d with $\mathrm{adm}_\sigma(u,d)$, every $P \in \mathrm{owed}_\sigma(u,d)$ reaches d (Definition 5), and $\delta(\mathrm{owed}_\sigma(u,d),d) = \delta(\mathrm{src}_\sigma(u),d)$.

(K) propagation edges order and block
$K_{\mathrm{prop}} \subseteq K_{\mathrm{order}}$ and $K_{\mathrm{prop}} \subseteq K_{\mathrm{block}}$.

The *retry invariant* of SPEC §13.7c fixes, for every outstanding unit, the corrected tuple stream, source sets, graph, scope and depth predicates, target baselines, and resolved source/target channel admission. The only permitted state changes are baseline tags written for packages that successfully release. Thus it entails the constancies used below; (H1) and (H2) name two historically problematic components, not a weaker substitute for the full invariant. One consequence the invariant states in its own terms is that $\mathrm{dlv}(,,)$ only grows (**G5**); Lemma 67 derives it.

(H1) and (H2) are explicit parts of the invariant. They are named because the 1.0.0 proof audit isolated them in counterexamples (proof obligations F140 and F143); they are not sufficient on their own. The fresh window of CCME 2.0.0 resolves the prerelease-discharge defect of proof obligation F139, while retries still require every other input listed above to remain fixed. Every theorem below states when it needs that full invariant.

(R) and (K) are not in SPEC. (R) holds automatically for a unit with a single source, since an admitted singleton is owed and reaches d by Item (A2), and for a target that has not released past c_u, where the owed set is the whole source set (Lemma 20) so that only the reaching clause is at issue. It can fail for a multi-source unit whose delivered source reaches the target and whose owed source does not, and Remark 99 shows what that costs. (K) holds for the default K_{order} and fails for the default K_{block}, which excludes `optionalDependencies` (Definition 3); Remark 87 shows what that costs. Both are flagged for the specification. The reference engine exhibited exactly this shape: in the build the campaigns of Section 10 ran, `1.11.0-rc.4+main.272772cb`, the bump axis credited a unit's whole source set to every dependent the walk reached, so a consumer two edges from one source and one from another was owed by both, and a consumer of only one of the packages a unit was written over was told it releases because of the other. The product repository's `main` fixes it at `e461d7d9` ("attribute a unit's sources to a dependent within the unit's depth of it", 22 September 2026, after the snapshot this report transcribes and unpublished when this was written), verified by the planner's differential test, the unit suite and the integration suite; the text of SPEC §9.2 still permits the shape, which is why (R) is a hypothesis here and not a theorem (proof obligation F132).

6.1 Lemmas

Lemma 65 (window antitonicity)**.** *Let $\sigma' = \mathrm{run}(\sigma, F)$ with $\mathrm{wf}(\sigma)$. For every $p \in \mathcal{P}$:*

$$W_{\sigma'}(p) = \begin{cases} \emptyset & \text{if } p \text{ published a non-prerelease version in the run,} \\ W_\sigma(p) & \text{otherwise,} \end{cases}$$

and

$$T_{\sigma'}(p) = \begin{cases} T_\sigma(p) \cup \{h\} & \text{if } p \in S(\sigma, F), \\ T_\sigma(p) & \text{otherwise.} \end{cases}$$

In particular $W_{\sigma'}(p) \subseteq W_\sigma(p)$, and along a run sequence $W_{\sigma_j}(p) \subseteq W_{\sigma_k}(p)$ for $j \geq k$. For the undelivered window, every successful release of p, stable or prerelease, gives $W^\circ_{\sigma'}(p) = \emptyset$; otherwise it is unchanged.

Proof. By Definition 25 the run adds only tags of the form $(p, \nu_\sigma(p), h)$ for $p \in S(\sigma, F)$, which is the claim about $T()$. If p is not among them, its tag set is unchanged, hence so are $\mathrm{scom}_\sigma(p)$ and its window. Suppose p is among them and $\nu_\sigma(p)$ is not a prerelease. Then $\nu_\sigma(p)$ exceeds $\mathrm{base}_\sigma(p)$ in SemVer precedence, and therefore exceeds $\mathrm{sbase}_\sigma(p)$: SPEC §13.9 computes it as $\mathrm{apply}(\mathrm{sbase}_\sigma(p), b)$ for the graduation and stable cases, and the guards `E185`, `E195`, `E153` and `E156` reject any computed or pinned version that fails to exceed the baseline. So the new tag is the highest-precedence non-prerelease tag of p, giving $\mathrm{scom}_{\sigma'}(p) = h$ and $W_{\sigma'}(p) = \mathrm{reach}(h) \setminus \mathrm{reach}(h) = \emptyset$. If $\nu_\sigma(p)$ is a prerelease, it is not a candidate for sbase() at all, so scom() and the train window are unchanged. Its tag is nevertheless the new baseline at h, so $W^\circ()$ is empty. The final claim follows by induction along the sequence. □

Remark 66. The third case of the proof, a prerelease release leaving the window untouched, is not an oversight in Definition 9 but its stated purpose (Remark 15). Admission nevertheless uses the empty undelivered window. Proof obligation F139 records the 1.0.0 defect that motivated this separation.

Lemma 67 (delivery is monotone)**.** *Let* $\sigma' = \mathrm{run}(\sigma, F)$ *with* $\mathrm{wf}(\sigma)$*. For all packages* P, D *and commits* $C \preceq h$*:*

(a) $\mathrm{dlv}_\sigma(P, C, D)$ *implies* $\mathrm{dlv}_{\sigma'}(P, C, D)$*;*
(b) if $h \notin T_{\sigma'}(D)$ *then* $\mathrm{dlv}_{\sigma'}(P, C, D)$ *iff* $\mathrm{dlv}_\sigma(P, C, D)$*;*
(c) if $h \in T_{\sigma'}(D)$ *then* $\mathrm{dlv}_{\sigma'}(P, C, D)$ *iff some* $t \in T_{\sigma'}(P)$ *has* $C \preceq t$*: iff* P *has released* C *at all.*

Consequently, under the retry invariant, $\mathrm{owed}_{\sigma'}(u, D) \subseteq \mathrm{owed}_\sigma(u, D)$ *for every unit* u*, with equality whenever* $h \notin T_{\sigma'}(D)$*; and along a run sequence* $\mathrm{dlv}_{\sigma_j}(P, C, D)$ *is non-decreasing in* j *and* $\mathrm{owed}_{\sigma_j}(u, D)$ *is non-increasing.*

Proof. (a): $T_\sigma(P) \subseteq T_{\sigma'}(P)$, and $\mathrm{basec}_{\sigma'}(D)$ is either $\mathrm{basec}_\sigma(D)$ or h, which reaches $\mathrm{basec}_\sigma(D)$; so a witness t for σ is one for σ'. (b): D gained no tag, so $\mathrm{basec}_{\sigma'}(D) = \mathrm{basec}_\sigma(D) =: b$ with $b \neq h$, and every tag the run added is at h; $h \preceq b$ would put h in $\mathrm{reach}(b)$, so $b \preceq h$ and acyclicity forbid it. Hence no added tag is a witness and the witness set is unchanged. (If D has no baseline both sides are false.) (c): $\mathrm{basec}_{\sigma'}(D) = h$ (Definition 8) and every release commit is an ancestor-or-self of h, so the second conjunct of Definition 10 is vacuous. The consequences restate (a) and (b) over $\mathrm{src}(u)$, which the retry invariant fixes, and induction along the sequence gives the last claim. □

Lemma 68 (outstanding-input stability)**.** *Assume the retry invariant. For every outstanding unit* u *and unreleased target* d*, its corrected tuple, source set, graph and scope/depth predicates, target baseline, resolved source and target channels, and channel-eligibility decision are the same in every retry.*

Proof. This is the retry invariant specialised to one unit and target. H1 prevents an inherited baseline channel from drifting; H2 prevents a successful source release from changing discharge and hence the source set. The remaining constancies are separate clauses of the invariant and cannot be derived from H1 and H2 alone. □

Lemma 69 (admission restriction)**.** *Under the retry invariant, admission of an outstanding unit* u *at a target* d *can change only through Item (A4), that is through* $\mathrm{owed}_\sigma(u, d)$*, which is non-increasing along the sequence. It becomes empty exactly when* d *is tagged at* h *and every source of* u *has released* c_u*.*

Proof. All other clauses of Definition 37 are fixed by the retry invariant (Lemma 68). The monotonicity is Lemma 67; the characterisation of emptiness is Lemma 67(b) (an untagged d keeps its owed set) and (c). □

Remark 70 (why H1 is part of the invariant). Without H1, the origin channel of an `inherit` unit can change when a source baseline moves from a train to stable. That can change an unreleased dependent's resolved channel and planned version between retries. Proof obligation F143 preserves the 1.0.0 counterexample; SPEC prevents it by fixing inherited baseline-channel state in the retry invariant.

Lemma 71 (a planned package tagged at h)**.** *Let $h \in T_\sigma(D)$ and $D \in \Pi(\sigma)$. Then* $\mathrm{dir}_\sigma(D) = \mathsf{none}$, $\mathrm{chan}_\sigma(D) = \mathrm{ch}(\mathrm{base}_\sigma(D))$, *$D$ has no applicable exact* `Release-As`*, and some unit u has* $\mathrm{adm}_\sigma(u, D)$. *For every such u and every $P \in \mathrm{owed}_\sigma(u, D)$:*

(i) no release of P reaches c_u; in particular $h \notin T_\sigma(P)$;
(ii) $c_u \in W^\circ_\sigma(P)$;
(iii) $\neg\mathrm{canc}(c_u, P)$, $\neg\mathrm{held}(P)$, $\mathrm{dir}_\sigma(P) \geq \mathrm{bump}(u) \neq \mathsf{none}$, *hence $P \in \Pi(\sigma)$;*
(iv) if $\mathrm{wf}(\sigma)$, *P precedes D in the publish order.*

Proof. $\mathrm{basec}_\sigma(D) = h$ gives $W^\circ_\sigma(D) = W_\sigma(D) \setminus \mathrm{reach}(h) = \emptyset$. Definition 38, both candidate sets of Definition 34 and the exact-version disjunct of Definition 21 all require a commit in $W^\circ_\sigma(D)$, so the first three claims hold and $D \in \Pi(\sigma)$ forces $\mathrm{prp}_\sigma(D) \neq \mathsf{none}$: some unit is admitted. (i): by Lemma 67(c), P owed means no $t \in T_\sigma(P)$ has $c_u \preceq t$; h reaches c_u. (ii): $\mathrm{scom}_\sigma(P)$ and $\mathrm{basec}_\sigma(P)$ are release commits of P when defined, so neither reaches c_u, and $c_u \preceq h$; by Definition 9 $c_u \in W^\circ_\sigma(P)$. (iii): by (ii) $\neg\mathrm{disch}_\sigma(P, c_u)$, so $P \in \mathrm{src}_\sigma(u)$ (Definition 19) forces $\neg(\mathrm{canc}(c_u, P) \vee \mathrm{held}(P))$. Then $P \in \mathrm{src}_\sigma(u) \cap \mathrm{scope}(u)$ and $c_u \in W^\circ_\sigma(P)$ put $\mathrm{bump}(u)$ into the join of Definition 38, and $\mathrm{bump}(u) \neq \mathsf{none}$ by Item (A1); an unheld package with a non-none bump is in the plan. (iv): $P \in \Pi(\sigma)$, $\mathrm{basec}_\sigma(D) = h$ and $\mathrm{owes}_\sigma(P, D)$; Definition 24 orders P before D. □

The lemma is the formal content of SPEC §19.3's "a dependent with no cause of its own": a package already tagged at h can be in the plan for one reason only, an owed provider that is itself in the plan, untagged, and ordered before it.

6.2 G1: termination

Theorem 72 (**G1**, termination)**.** *Every traversal of Definition 28 halts after at most $|\mathcal{P}|$ insertions into* `seen` *and $O(|\mathcal{P}| + |E|)$ work, regardless of the declared depth, including ∞ (`+*`), and regardless of whether the graph is acyclic. Propagation performs at most $2|\mathcal{U}|$ such traversals, so it halts, in $O(|\mathcal{U}| \cdot (|\mathcal{P}| + |E|))$ traversal work.*

Proof. A package is inserted into `seen` only after a test that it is absent, and `seen` is never cleared, so there are at most $|\mathcal{P}|$ insertions. Only inserted packages (and the initial sources, inserted at seeding) ever enter `next` and hence `frontier`, so each package is expanded at most once, and the edges scanned across the whole traversal are therefore at most $\sum_p |\mathrm{dep}(p)| = |E|$. The loop terminates either by exhausting ℓ iterations or, when $\ell = \infty$, by the empty-frontier break, which must occur within $|\mathcal{P}|$ iterations since each non-empty frontier consumes at least one distinct unvisited package. No step of this argument uses acyclicity: a cycle merely means some edge leads to a package already in `seen`, and that edge is skipped. The outer loops of SPEC §9.2 run over the finite set $\mathcal{U}$ once per axis; the per-target work of phase 3 (the owed set and the resolvability test) is finite and adds no traversal. □

Remark 73. Acyclicity (Definition 3) is therefore not what makes propagation terminate; `seen` is. Acyclicity is needed for the publish order (Proposition 45), not here. This matters because it means an implementation that loosens the `E200` check does not silently gain a non-terminating traversal; it gains an unorderable publish phase, which is a different and louder failure.

6.3 G2: completeness

Theorem 74 (**G2**, completeness: no orphans)**.** *Assume the retry invariant. If* $\mathrm{adm}_{\sigma_1}(u, D)$*, then for every* j *either* $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ *or* $\mathrm{adm}_{\sigma_j}(u, D)$*, and in the latter case* $\mathrm{prp}_{\sigma_j}(D) \geq \mathrm{pb}(u)$*. Moreover* $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ *holds only if* D *is tagged at* h *in* σ_j *and every* $P \in \mathrm{src}_\sigma(u)$ *has a release reaching* c_u*. A release by a source alone does not discharge the contribution, and neither does a release by* D *at a commit reaching no release of some source carrying* c_u*.*

Proof. Take the conjuncts of Definition 37 in turn. Item (A1) depends only on the unit. Item (A3) and Item (A5) depend only on h and the configuration. Item (A2) is $D \in R(\mathrm{src}_{\sigma_j}(u), \mathrm{pd}(u))$, and by the retry invariant the source set is unchanged, as are the graph and the depth, so the traversal is literally the same computation. Item (A6) is constant by Lemma 68. Item (A4) is the one clause that varies, and it is exactly the disjunction in the statement. So whenever $\mathrm{owed}_{\sigma_j}(u, D) \neq \emptyset$ every conjunct holds, and $\mathrm{prp}_{\sigma_j}(D)$, a join over the admitted units (Definition 38) with u among them, is at least $\mathrm{pb}(u)$.

For the "moreover": $\mathrm{owed}_{\sigma_1}(u, D) \neq \emptyset$ by Item (A4) at σ_1, so some $P \in \mathrm{src}_\sigma(u)$ has gone from undelivered to delivered between σ_1 and σ_j. If $h \notin T_{\sigma_j}(D)$, Lemma 67(b) applied along the sequence says nothing changed; so D is tagged at h in σ_j, and then Lemma 67(c) makes $\mathrm{owed}_{\sigma_j}(u, D) = \emptyset$ equivalent to every source of u having a release reaching c_u. The two final sentences are the contrapositive read against the same lemma: a source's release leaves $h \notin T(D)$ untouched, and a release of D at h with some source still lacking a release reaching c_u leaves that source owed. □

The theorem is the formal content of "there are no orphans", and it now covers both shapes of SPEC §13.7a. A provider that publishes while a consumer's publish fails leaves the consumer's owed set intact (Lemma 67(b)), so the consumer is planned again until it releases. A consumer that proceeds past the commit on a cause of its own while its provider fails is tagged at h with the provider unreleased; by Lemma 67(c) the provider is still owed, and the consumer is planned again, ordered after the provider (Lemma 71), until both have released. Note where the proof would break under the readings of Item (A4) that Example 39 contrasts. Against the source's window, the provider's successful release would empty *that* window, the unit would cease to be a propagation source, and the waiting consumer would never be released. Against the target's window, the proceeding consumer's own release would empty *that* window, and the consumer that got ahead would never be planned again. One membership test, evaluated against the right relation, is the whole difference between a resumable release and a silently lossy one.

Remark 75 (the discharge that is not a delivery). The "moreover" clause is satisfied, and the contribution leaves the plan, in one situation where nothing was delivered: D already tagged at h when the sequence starts, its owed provider P publishing at h in a run in which D then fails. Proposition 26 describes it; the run reports `E201` and the theorem does not claim otherwise. Where D is not tagged at h in σ_1 the situation cannot arise, since then $h \notin T_{\sigma_j}(D)$ until D itself releases in the sequence.

6.4 G3: version stability

Theorem 76 (**G3**, version stability)**.** *Assume the retry invariant. If $d \in \Pi(\sigma_1)$, d is not released in runs $1..k-1$, and no run $i < k$ reports* `E201` *for d (Definition 25), then $d \in \Pi(\sigma_k)$ and $\nu_{\sigma_k}(d) = \nu_{\sigma_1}(d)$. The third hypothesis is vacuous when $h \notin T_{\sigma_1}(d)$.*

Proof. First, $\text{owed}_{\sigma_k}(u, d) = \text{owed}_{\sigma_1}(u, d)$ for every unit u. If $h \notin T_{\sigma_1}(d)$ then, d having gained no tag, $h \notin T_{\sigma_k}(d)$ and Lemma 67(b) gives equality run by run; this is also why the third hypothesis is then vacuous, since reporting `E201` for d requires $h \in T_{\sigma_i}(d)$. If $h \in T_{\sigma_1}(d)$, then $\text{basec}_{\sigma_i}(d) = h$ throughout, and by Lemma 67(c) the owed set can shrink only when some $P \in \text{owed}_{\sigma_i}(u, d)$ receives a tag at h in run $i < k$, that is $P \in S(\sigma_i, F_i)$. Since $\text{owed}_{\sigma_i}(u, d) \neq \emptyset$ and the other clauses are constant, $\text{adm}_{\sigma_i}(u, d)$, so $\text{owes}_{\sigma_i}(P, d)$. Run i published something, so it was not refused and $\text{wf}(\sigma_i)$ holds (Definition 25); hence $d \in \Pi(\sigma_i)$ after P. d was not released in run i, and it was not blocked, because $P \notin X_d$ gives it a cause of its own (Definition 23); so $d \in F_i$, and run i reports `E201` for d, which the hypothesis excludes.

Now d gained no tag, so $\text{base}_{\sigma_k}(d) = \text{base}_{\sigma_1}(d)$ and $\text{sbase}_{\sigma_k}(d) = \text{sbase}_{\sigma_1}(d)$, and by Lemma 65 its windows are unchanged. Then $\text{dir}_{\sigma_k}(d) = \text{dir}_{\sigma_1}(d)$, because Definition 38 reads the window, $\text{scope}(u)$, and $\text{src}_\sigma(u)$, the last constant by the retry invariant. And $\text{prp}_{\sigma_k}(d) = \text{prp}_{\sigma_1}(d)$: every clause of Definition 37 is constant for every unit, Item (A4) by the first paragraph and the rest by Lemma 68. Hence $\text{eff}_{\sigma_k}(d) = \text{eff}_{\sigma_1}(d)$. Its resolved and baseline channels, changed-provider status, and applicable exact `Release-As` cause are also fixed by the retry invariant. Thus whichever disjunct of Definition 21 put d in the initial plan still holds. For a fixed version function, SPEC §13.9 reads unchanged stable and current baselines, resolved channel, applicable exact version, and corrected train records. Those inputs are fixed for an unreleased target by the retry invariant, so its result is unchanged. This proves replay stability. The graduation core itself follows the specific train-target rule of SPEC §11.5; proof obligation F134 records the inconsistent SPEC §13.9 summary rather than treating that intent as absent. □

Remark 77 (the prerelease case survives, for a different reason)*.* For a target d planned on a prerelease channel, the version is core(·)-plus-counter per SPEC §11.4, and the counter is read from $\text{base}_\sigma(d)$, which is unchanged precisely because d was *not* released. This preserves the counter, but extending Theorem 76 to prerelease targets still requires invariant resolved channels and admitted contributions, including H1's channel-stability component. The prerelease problem of proof obligation F139 is about packages that *were* released, and is a convergence failure, not a stability failure.

Theorem 76 is as much an operational property as a formal one: after a partial failure, the rerun publishes the same numbers the operator already reviewed. It is what makes a resumed release a continuation rather than a fresh decision. It says nothing about a package that *was* released: a consumer that proceeded is planned again at a patch on its new baseline (`cli@2.1.1` in Example 39), which is the version its catch-up is owed, not the one it was first planned at.

Example 78 (four theorems in two tables)**.** Table 1 continues the first shape of Example 39 through an adversarial schedule: run 1 loses `cli`, run 2 loses it again, run 3 succeeds. Table 2 runs the second shape, where `cli` proceeds. Each column is a theorem. `docs-site` and `@acme/theme` never appear, at any distance 2, in any run: the unit said `^`, and Theorem 81 forbids a later run from reconsidering that.

Run	State σ	Plan $\Pi(\sigma)$	Published	Empty?
1	σ_0 (Example 4)	`core` 1.5.0, `cli` 2.0.1, `ui` 0.9.2, `api` 1.2.1	`core`, `ui`, `api`; `cli` fails	no
2	$\sigma_1 = \sigma_0 \cup 3$ tags	`cli` 2.0.1	nothing; `cli` fails again	no
3	$\sigma_2 = \sigma_1$	`cli` 2.0.1	`cli`	no
4	$\sigma_3 = \sigma_2 \cup \{$`cli@2.0.1`$\}$	$\emptyset$	(none)	yes

Table 1: The waiting consumer of Example 78. Theorem 74: `cli` is in every plan until it publishes, its owed set {`core`} unchanged by Lemma 67. Theorem 76: it is at `2.0.1` in all of them. Theorem 79: `core`, `ui` and `api` leave after run 1. Theorem 81: nothing new ever enters. Theorem 85: the plan shrinks to empty. Run 2 publishes nothing, which Theorem 85 treats as the hard-error case rather than the partial-failure one.

Run	State σ	Plan $\Pi(\sigma)$	Published	Progress?
1	σ_0, C carrying u and u'	`core` 1.5.0, `cli` 2.1.0, `ui` 0.9.2, `api` 1.2.1	`cli` proceeds; `core` fails; `ui`, `api` blocked	yes
2	$\sigma_1 = \sigma_0 \cup \{$`cli@2.1.0`$\}$	`core` 1.5.0, `cli` 2.1.1, `ui` 0.9.2, `api` 1.2.1	all four	yes
3	$\sigma_2 = \sigma_1 \cup 4$ tags	$\emptyset$	(none)	

Table 2: The proceeding consumer of Example 78, SPEC vector 80d. Theorem 74: `cli` stays admitted after its own release because `core` has not delivered C to it. Theorem 76: `ui` and `api` keep their numbers. Theorem 79: after run 2 nothing is owed. Theorem 85: run 1 published a package and the plan did not shrink; what it did was tag `cli` at h for the first time, which is the measure the proof uses.

6.5 G4: no double release

Theorem 79 (**G4**, no double ledger delivery)**.** *Assume the retry invariant. If D is released in run k and, after run k, every $P \in \mathrm{src}_\sigma(u)$ has a release reaching c_u, then $\mathrm{adm}_{\sigma_j}(u, D)$ is false for every $j > k$.*

Proof. $h \in T_{\sigma_{k+1}}(D)$, so by Lemma 67(c) every source of u has delivered c_u to D and $\mathrm{owed}_{\sigma_{k+1}}(u, D) = \emptyset$. The owed set is non-increasing (Lemma 67), so it stays empty and Item (A4) fails at every later σ_j. The full train window may retain c_u for aggregate prerelease version arithmetic; it cannot re-admit the contribution. □

A D that proceeded past c_u before a source released it does not meet the hypothesis: that source still lacks a release reaching c_u, D is admitted again (Lemma 67(c)), and it is bumped once more when the release comes. That later release is the delivery, not a second one. The sufficient condition of CCME 2.0.0, "released at a commit whose ancestry contains c_u", is no longer sufficient on its own, and Example 39 shows why it must not be.

Corollary 80 (exactly-once ledger discharge)**.** *Under the retry invariant, a contribution admitted to D remains admitted until D is tagged at h with every source of u having released c_u (Theorem 74), and is never re-admitted after that (Theorem 79). If the sequence delivers it, it is therefore discharged in the tag ledger exactly once. D may be published more than once meanwhile, a consumer that proceeded being the case, but the discharging release is the last of those. The single exception is the reported loss of Proposition 26, where the ledger records a discharge that no release of D performed.*

This is the idempotent-consumer property of the saga, obtained without a deduplication table: the git tags are the record, and delivery is the query against them. This is not an exactly-once guarantee for external side effects. A publish can succeed before its tag is recorded; recovery across that gap requires the verified adoption of SPEC §19.4 or an idempotent publisher. The ledger proof alone cannot prevent a repeated external operation.

6.6 G5: no blast-radius widening

Theorem 81 (**G5**, no blast-radius widening)**.** *Assume the retry invariant. For every unit u and every $j \geq 1$,*

$$\{\, d : \mathrm{adm}_{\sigma_j}(u, d) \,\} \;\subseteq\; \{\, d : \mathrm{adm}_{\sigma_1}(u, d) \,\}.$$

Proof. Compare the conjuncts of Definition 37, as in Theorem 74, but now in the direction of restriction. Items (A1), (A3) and (A5) are constant. Item (A2) is constant by the retry invariant: the traversal has the same source set, the same depth and the same graph, all fixed at h. Item (A6) is constant under the retry invariant. Item (A4), $\mathrm{owed}_{\sigma_j}(u, d) \neq \emptyset$, is antitone in j by Lemma 67: $\mathrm{dlv}(,,)$ only grows. A conjunction of constant and antitone predicates is antitone. □

Remark 82 (why this is a security property). SPEC §18.1 depends on Theorem 81. If the admitted set could grow between runs, an attacker able to induce a publish failure (by exhausting a registry quota, say) could enlarge the set of packages a commit releases *after* that commit had been reviewed and merged. The theorem says the achievable effect of inducing a failure is confined to delaying a subset of what was already going to be released. Delivery preserves this: a target it admits that the window test would not is one that released past c_u on its own account while within the unit's reach, and the unit's traversal, scope and depth are what they were, so nothing outside the unit's declared blast radius is reached.

Lemma 83 (the plan is non-increasing)**.** *Assume the retry invariant. Then $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$ for every j, hence $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$.*

Proof. Take $p \in \Pi(\sigma_{j+1})$; $\mathrm{held}(p)$ is constant, so p is not held at σ_j either. If p was released in run j, then $W^{\circ}_{\sigma_{j+1}}(p) = \emptyset$ (Lemma 65) and by Lemma 71 its only cause is a unit u with $\mathrm{adm}_{\sigma_{j+1}}(u, p)$; by Theorem 81 $\mathrm{adm}_{\sigma_j}(u, p)$, so $\mathrm{prp}_{\sigma_j}(p) \neq \mathsf{none}$ and $p \in \Pi(\sigma_j)$. If p was not released, its windows and tags are unchanged, so $\mathrm{dir}_{\sigma}(p)$ and the exact-version disjunct are unchanged, $\mathrm{prp}_{\sigma}(p)$ can only have shrunk (Theorem 81 applied to each unit), and $\mathrm{chan}_{\sigma}(p)$ and $\mathrm{ch}(\mathrm{base}_{\sigma}(p))$ are unchanged under the retry invariant; so every disjunct of Definition 21 that holds at σ_{j+1} held at σ_j. □

What the lemma no longer gives is strict decrease. A package released in run j can be in $\Pi(\sigma_{j+1})$: it proceeded past a commit its provider has not delivered. Theorem 85 therefore counts something else.

Lemma 84 (well-formedness is preserved)**.** *Assume the retry invariant, (R), and $K_{\mathrm{prop}} \subseteq K_{\mathrm{order}}$. If $\mathrm{wf}(\sigma_j)$ then $\mathrm{wf}(\sigma_{j+1})$. Hence every state of a run sequence is well-formed and no run of it is refused.*

Proof. Let $P \in \Pi(\sigma_{j+1})$, $\mathrm{basec}_{\sigma_{j+1}}(D) = h$ and $\mathrm{owes}_{\sigma_{j+1}}(P, D)$ through a unit u. By Theorem 81 $\mathrm{adm}_{\sigma_j}(u, D)$, and by Lemma 67 $P \in \mathrm{owed}_{\sigma_j}(u, D)$, so $P \in \Pi(\sigma_j)$ (Lemma 83) and $\mathrm{owes}_{\sigma_j}(P, D)$. Since $\mathrm{owed}_{\sigma_{j+1}}(u, D) \neq \emptyset$ and $\mathrm{held}(D)$ is constant, $D \in \Pi(\sigma_{j+1})$ once $D \in \Pi(\sigma_j)$ or $\mathrm{prp}_{\sigma_{j+1}}(D) \neq \mathsf{none}$; the latter holds outright. For the order: by (R) P reaches D over K_{prop},

hence over K_{order}, and the topological order of Definition 44 places P before D. The second sentence is induction from $\text{wf}(\sigma_1)$ (Definition 27). □

6.7 G6: convergence

Say that run j *makes progress* when it publishes a package p with $h \notin T_{\sigma_j}(p)$: a package tagged at h for the first time.

Theorem 85 (**G6**, convergence)**.** *Assume the retry invariant, and suppose every run whose plan is non-empty makes progress. Let $n = |\Pi(\sigma_1)|$. Then $\Pi(\sigma_{n+1}) = \emptyset$.*

A refused run (Definition 25) publishes nothing, so the hypothesis entails that no run with a non-empty plan is refused; Lemma 84 says when that is automatic.

Proof. First, $\Pi(\sigma_{j+1}) \subseteq \Pi(\sigma_j)$, so $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$ for every j (Lemma 83).

Let $\mu_j = |\{\, p \in \Pi(\sigma_1) : h \notin T_{\sigma_j}(p) \,\}|$, the planned packages not yet tagged at h. Tags only accumulate, so μ is non-increasing, and $\mu_1 \le n$. If run j makes progress, the package it tags for the first time lies in $\Pi(\sigma_j) \subseteq \Pi(\sigma_1)$, so $\mu_{j+1} < \mu_j$. And if $\Pi(\sigma_j) \neq \emptyset$ then $\mu_j \ge 1$: take $p \in \Pi(\sigma_j)$; either $h \notin T_{\sigma_j}(p)$, or by Lemma 71 there is $P \in \Pi(\sigma_j) \subseteq \Pi(\sigma_1)$ with $h \notin T_{\sigma_j}(P)$. So while the plan is non-empty each run strictly decreases a quantity that is at least 1 and at most n; the plan is therefore empty by run $n+1$, and once empty it stays empty by the first paragraph. □

The old measure, $|\Pi(\sigma_j)|$, fails here: in Table 2 run 1 publishes `cli` and $|\Pi(\sigma_2)| = |\Pi(\sigma_1)|$. The bound n survives because every package is tagged at h for the first time at most once, and a plan cannot be non-empty without containing such a package. The bound is restricted to the invariant domain. Publishing a suppressed source or changing channel admission may expose a finite follow-up obligation; SPEC §19.6 requires re-planning and renewed review, not an unconditional global-convergence claim. Held packages remain excluded.

Proposition 86 (when publishing anything is progress)**.** *Assume the retry invariant, (R) and $K_{\text{prop}} \subseteq K_{\text{block}}$. Then every well-formed run that publishes at least one package makes progress. Under these hypotheses Theorem 85 therefore holds with its hypothesis weakened to "every run whose plan is non-empty publishes at least one package".*

Proof. Let run j publish a non-empty set, and let p be the first published package in the publish order. Suppose $h \in T_{\sigma_j}(p)$. By Lemma 71, p's only causes are units u with $\text{adm}_{\sigma_j}(u, p)$, and every $P \in \text{owed}_{\sigma_j}(u, p)$ is in $\Pi(\sigma_j)$, untagged at h, with $\text{dir}_{\sigma_j}(P) \neq \mathsf{none}$, and precedes p in the order. No such P was published, since p is the first; none was blocked, since a direct bump is a cause of its own; so every such P failed and lies in X_p. Then $\text{owed}_{\sigma_j}(u, p) \subseteq X_p$ for every admitted u, and with the first three disjuncts of Definition 23 absent, $\neg\text{own}_{\sigma_j}(p, X_p)$. By (R) some owed P reaches p over K_{prop}, hence over K_{block}, so p transitively depends on a member of X_p and is blocked: a contradiction with its being published. Hence $h \notin T_{\sigma_j}(p)$ and the run makes progress. □

Remark 87 (a non-blocking propagation edge). Under the default configuration K_{prop} contains `optionalDependencies` and K_{block} does not, so (K) fails and Proposition 86 does not apply. The weaker hypothesis then does not suffice. Let p optionally depend on P, let one commit carry `feat(P)^`, and let P fail on every run. In run 1, p is not blocked, since the edge is not in K_{block}, and it has no cause of its own; it proceeds anyway, is reconciled to P's old baseline, and is tagged at h. In run 2 it is owed P's release (Lemma 67(c)), is planned after P, and,

P failing again, is published again with nothing new. Every run publishes a package; no run makes progress; the plan never empties. The count n of Theorem 85 bounds the runs that make progress, not the runs that publish, and a spec-level reading of "successful-progress run" must say so. The republished p carries nothing its predecessor lacked, which is a second cost of the same configuration. Both are flagged for the specification. The reference implementation does not exhibit the scenario: it blocks over every declared dependency kind and treats an update from a failed provider as no cause (proof obligation F135), so the caught-up consumer of vector 80d is planned again each run and blocked with `W194` until its provider publishes, each such run exiting non-zero; the remark concerns the specification's default kinds, not the engine.

Proposition 88 (universal tagging is necessary, SPEC §13.10a)**.** *Assume the retry invariant. Suppose the protocol were weakened so that a released package p whose publish target uploads no artefact is versioned and has its manifest written, but is not tagged. Then if* $\mathrm{eff}_{\sigma_1}(p) \neq \mathsf{none}$*,* $p \in \Pi(\sigma_j)$ *for every j, and Theorem 85 fails.*

Proof. p gains no tag in any run, so $h \notin T_{\sigma_j}(p)$ for all j; by Lemma 65 $W^{\circ}_{\sigma_j}(p) = W^{\circ}_{\sigma_1}(p)$, and by Lemma 67(b) $\mathrm{owed}_{\sigma_j}(u,p) = \mathrm{owed}_{\sigma_1}(u,p)$ for every unit. Hence $\mathrm{dir}_{\sigma_j}(p) = \mathrm{dir}_{\sigma_1}(p)$ and, every clause of Definition 37 being constant, $\mathrm{prp}_{\sigma_j}(p) = \mathrm{prp}_{\sigma_1}(p)$. The plan's first disjunct holds at every σ_j, so $\mu_j \geq 1$ for all j in the proof of Theorem 85 and no run empties the plan. □

This is why SPEC §13.10a tags whatever is released, including private, internal and artefact-less packages, and why the convergence check of SPEC §19.6 (`E199`) admits exactly one exclusion, held packages. A permanent exception to convergence would not be a caveat to document; it would disable the one check that detects the orphan failure of SPEC §13.7a, and disable it precisely for the internal-only packages where nobody is watching a registry.

6.8 G7: channel discharge

Theorem 89 (**G7**, channel discharge)**.** *Suppose d is released in run k on channel ch, so that* $\mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$ *for $j > k$. Then for every unit u and every $j > k$,* $\mathrm{pcf}_{\sigma_j}(u,d) \neq ch$*: the channel d has arrived on is never proposed for it again.*

Proof. Write $cur = \mathrm{ch}(\mathrm{base}_{\sigma_j}(d)) = ch$; this is read from d's tag, not from anything computed in the run. Every branch of Definition 32 ends in the test "if $target = cur$, return $\bot$". A result of ch would require $target = ch = cur$, which that test rejects. The transition branch fails one step earlier where it applies: having arrived at t, d no longer matches an $f \neq t$, and for $f = *$ with $t = \mathsf{stable}$ the match requires $cur \neq \mathsf{stable}$, which fails. □

Corollary 90 (channel-axis ledger discharge)**.** *After any release of d, $W^{\circ}()$ excludes every delivered channel directive (Lemma 65), and Definition 33 admits none of them. The no-op and transition checks of Theorem 89 independently prevent re-proposing the channel already recorded in the baseline. These are complementary checks; the baseline comparison is not a substitute for fresh-work admission.*

The two axes discharge differently. The channel axis discharges through the target's undelivered window, with the no-op check on top; the bump axis discharges through delivery (Theorem 79), which on a target that has not released past the commit is the same window and on one that has is stricter. Under the retry invariant both survive a partial failure unchanged. This also marks the boundary of what Theorem 89 gives. It says the channel d

arrived on is not re-proposed; it does *not* say nothing is proposed. The historical CCME 1.0.0 counterexample in proof obligation F139 used its unsplit window to let a second, older directive propose a different channel on every run. The fresh-window gate, normative since CCME 2.0.0, discharges that directive after release.

6.9 G8: axis independence

Theorem 91 (**G8**, axis independence)**.** *The channel assignment* $\mathrm{chan}_\sigma(\cdot)$ *is a function of* σ*,* h *and the configuration that does not read any quantity computed by phase 3. Consequently, altering the bump directives* $\mathrm{pb}(u), \mathrm{pd}(u), \mathrm{ps}(u)$ *of any set of units leaves* $\mathrm{chan}_\sigma(\cdot)$ *unchanged, and in particular a* `W208` *suppression removes no channel assignment and causes no package to lose a channel change it would otherwise have had.*

Proof. By Definition 33, phase 1 reads the units' channel directives, $\mathrm{src}_\sigma(u)$, the traversal, $W^\circ_\sigma(\cdot)$, $\mathrm{canc}(\cdot,\cdot)$, and, through Definition 32, $\mathrm{base}_\sigma(\cdot)$. By Definition 34, phase 2 reads phase 1's output and the same fixed quantities. None of these is a phase-3 output: $\mathrm{prp}_\sigma(\cdot)$, $\mathrm{eff}_\sigma(\cdot)$, the admission predicate and $\mathrm{owed}_\sigma(\cdot,\cdot)$ appear nowhere in Definitions 32 to 34. So $\mathrm{chan}_\sigma(\cdot)$ is determined before phase 3 runs and is invariant under any change confined to phase-3 inputs. □

The dependency between the axes is thus a single directed edge, from channels to bumps, and Theorem 91 is the statement that there is no edge back. Two operational consequences follow, both worth stating because they are what a plan reader needs: a `W208` in a plan means a caret did not reach, not that a channel directive failed; and a package may legitimately appear in a plan with a channel change and no bump (`W202`), or with a bump and no channel change, and neither shape indicates a defect.

Remark 92 (the converse does not hold)*.* CCME 1.0.0 stated **G8** as "suppressing a bump under SPEC §9.3a never suppresses a channel, *and vice versa*". SPEC states only the one-way result above. The old proof observed that the passes share only $\mathrm{chan}_\sigma(\cdot)$, which phase 3 reads and never writes. That argument establishes exactly the direction proved above. The converse is false, and intentionally so: a channel suppressed by `W200` leaves $\mathrm{chan}_\sigma(d)$ at its baseline value, and a bump from sources on another channel is then suppressed by Item (A6). Proof obligation F142 states the correction.

7 Determinism, idempotency, resumption

SPEC §17.2 states three obligations that are usually treated as informal quality properties. They are provable in this model, and two of them are corollaries of Section 6.

Theorem 93 (determinism)**.** $\Pi(\sigma)$*, the versions* $\nu_\sigma(\cdot)$*, the planned publish order and the ordered planning diagnostic list are functions of* $(\sigma, h, \mathrm{config})$ *alone. Two conforming implementations, on two machines, at two times, produce identical plan output. Concurrent task completion, logs, and other operational events are outside this claim.*

Proof. Every quantity in Sections 2 and 3 is defined by a closed expression in σ, h and the configuration: Definitions 9 and 19 from ancestry and tags, Definition 10 and the owed windows of Definition 16 from the same; Definition 16 from commit messages and the configured types, separator and limits; Definition 28 from the graph at h; Definitions 32 to 35 and 37 from those; Definitions 21 and 44 from those. None mentions wall-clock time, commit dates, tag creation order, or the run history.

Three points admit a choice and each is resolved by a stated total order. Conflicting propagated channels and disagreeing origin channels resolve to the byte-wise least package name (`W160`, Definition 32). Competing directives resolve by SPEC §11.6 precedence: newest commit first, then last unit within the commit, a total order on units, since commits carry a topological index and units an index within their commit. Publish-order ties resolve by byte-wise least name (Proposition 45). SPEC §17.2 additionally requires every collection reaching the output to be sorted by package name, then commit topological index, then unit index, so no unordered container's iteration order is observable. A function of fixed inputs with all ties broken by total orders is single-valued. □

Theorem 94 (resumption after partial failure)**.** *Assume the retry invariant and* $\mathrm{wf}(\sigma_1)$*. Let* $\sigma_2 = \mathrm{run}(\sigma_1, F)$ *with blocked set* $B = B(\sigma_1, F)$ *and published set* $S(\sigma_1, F)$*, and suppose the run reports* `E201` *for no package. Then*

$$\Pi(\sigma_2) \;=\; F \cup B \cup A, \qquad A = \{\, p \in S(\sigma_1, F) \;:\; \mathrm{owes}_{\sigma_2}(P, p) \text{ for some } P \in F \cup B \,\},$$

and $\nu_{\sigma_2}(p) = \nu_{\sigma_1}(p)$ *for every* $p \in F \cup B$*.*

Proof. $\subseteq$: $\Pi(\sigma_2) \subseteq \Pi(\sigma_1)$ by Lemma 83, and every member of $\Pi(\sigma_1)$ lies in F, in B or in $S(\sigma_1, F)$. Take $p \in \Pi(\sigma_2) \cap S(\sigma_1, F)$. It is tagged at h in σ_2, so by Lemma 71 its only cause is a unit u with $\mathrm{adm}_{\sigma_2}(u, p)$, and every $P \in \mathrm{owed}_{\sigma_2}(u, p)$ is in $\Pi(\sigma_2) \subseteq \Pi(\sigma_1)$ and untagged at h, hence not published in run 1, hence in $F \cup B$; so $p \in A$. $\supseteq$: each $p \in F \cup B$ was planned in run 1 and not released, and the run reported `E201` for nothing, so Theorem 76 with $k = 2$ gives $p \in \Pi(\sigma_2)$ at the same version. Each $p \in A$ has an admitted unit with a non-empty owed set, so $\mathrm{prp}_{\sigma_2}(p) \neq \mathsf{none}$, and $\mathrm{held}(p)$ is constant and false; so $p \in \Pi(\sigma_2)$. □

The set A is new with delivery: the consumers that proceeded past a commit on a cause of their own while a provider failed or was blocked, and are owed that provider's release. They return to the plan at a version their catch-up computes, not at the version they were first planned at (Theorem 76 says nothing about a released package), and they are ordered after the provider (Lemma 71). Under the window test of CCME 2.0.0, A was empty, because a published package could not be planned again at the same h; that is the orphan of Example 39, second shape.

Theorem 94 is the exact statement of what the operator is promised after a failed run under the retry invariant: the next run does the rest of the same job, at the same numbers for what never released and at the owed catch-up for what proceeded, without coordinator state to clean up. An ambiguous external publish outcome still requires SPEC §19.4 reconciliation. It is also what SPEC §19.6 checks. After a run in which nothing failed or was blocked, $F \cup B = \emptyset$, so $A = \emptyset$ and the re-planned set must be empty; a non-empty one is `E199`. Inside the retry invariant it contradicts Theorem 94; outside it, SPEC §19.6 reports the changed admission inputs for renewed review.

Corollary 95 (idempotency)**.** *Running the engine twice with no intervening commits and no publish yields the same plan. Under the retry invariant, running it after a fully successful publish from a well-formed state yields the empty plan.*

Proof. The first is Theorem 93: the state is unchanged, so the plan is. The second is Theorem 94 with $F = \emptyset$: then $B = \emptyset$, no `E201` is reported (it needs a failed package), and $A = \emptyset$ because nothing is left to owe; so $\Pi(\sigma_2) = \emptyset$. Equivalently, every $p \in \Pi(\sigma_1)$ is tagged at h in σ_2 together with every package it was owed by, so Lemma 71 leaves it no cause; the invariant excludes both input drift and newly visible catch-up. □

Remark 96 (what determinism forbids). Theorem 93 is the reason SPEC §13.11 may permit optimisation of the computation but not of its result. An implementation may cache identical traversals, invert loops, cache lexical parses and publish concurrently; it may not introduce parallelism whose scheduling affects the iteration order of any collection that reaches the output. Section F9 records the precise conditions under which such transformations preserve those observables.

8 The staleness dual

SPEC §9.2 walks *down* from the packages a commit changed. An audit command wants the opposite walk: given a package, which of its dependencies is it behind, and by how much? SPEC §13.7b gives that walk and asserts that the two agree, calling disagreement an implementation bug. This section proves that the audit is sound against SPEC §9.2 unconditionally, that it is complete under two conditions the current text does not state, and exhibits the disagreements outside them. The pseudo-code is reproduced from SPEC §13.7b unchanged, except that the arrow and ellipsis glyphs in the comment on `dist`, which the monospace font lacks, are set as `->` and `....`

```
staleSources(D):
    edges = graph.restrictedTo(config.propagation.kinds)  # ğ8.4: the same edges as ğ9.2
    channel = resolveChannelsForAudit()                    # same phase-2 result as ğ9.2
    dist  = shortestPathsUp(D, edges)   # P -> edge count of the shortest path P -> ... -> D
    out   = {}
    for u in units:                                        # every retained tuple, ğ13.4
        if bumpOf(u) == none:                              continue
        b = (u.propagate == 'inherit') ? bumpOf(u) : u.propagate    # as ğ9.2
        if b == none or u.depth == 0:                      continue
        sources = sourcePackages(u)                        # ğ13.4a
        if sources is empty:                               continue
        if D in sources:                                   continue   # ğ9.2 seeds seen = sources
        sources = owed(u, D)                               # ğ13.4a: what D is still owed
        if sources is empty:                               continue   # every source delivered
        reaching = { P in sources : P in dist }
        if reaching is empty:                              continue
        if not resolvableBy(sources, D, channel):     continue   # ğ9.3a
        level = min({ dist[P] for P in reaching })         # measured from the whole source set
        if level > u.depth:                                continue
        if D not in resolve(u.propagateScope):             continue
        if cancelledFor(commitOf(u), D):                   continue
        for P in reaching:
            out |= { (P, u, level, b) }
    return out
```

Three things changed from the CCME 2.0.0 audit. The loop runs over every retained tuple, not over the units of $W^{\circ}(D)$: a commit D has released past can still be owed to it by a source that released it later, and iterating the target's window alone misses exactly the consumers that got ahead of a provider, which are the ones the audit exists to find after a partial failure. `sources` is rebound to $\mathrm{owed}(u, D)$ before the distance and resolvability tests, so the rows name the owed sources, as `prov[D]` does in SPEC §9.2. And `resolvableBy` is tested before the level test.

Definition 97. $\mathrm{stale}_{\sigma}(d) = \{(p, u, \ell, b)\}$ is the set the procedure returns. For the rows of a unit u, p ranges over the owed sources of u that reach d, $\ell = \delta(\mathrm{owed}_{\sigma}(u, d), d)$, the least distance from an owed source, and $b = \mathrm{pb}(u)$. Write $\mathrm{sch}_{\sigma}(u)' = \{\mathrm{chan}_{\sigma}(P) : P \in \mathrm{owed}_{\sigma}(u, d)\}$ for the channel set the audit tests, and $\mathrm{rsv}_{\sigma}(u, d)'$ for Definition 35 evaluated over it.

Theorem 98 (duality)**.** *For every state σ and package d:*

(a) (soundness) if u contributes a row to $\mathrm{stale}_\sigma(d)$ *then* $\mathrm{adm}_\sigma(u,d)$*; hence* $\mathrm{stale}_\sigma(d) \neq \emptyset$ *implies* $\mathrm{prp}_\sigma(d) \neq \mathtt{none}$ *and* $\bigsqcup\{\, b : (p,u,\ell,b) \in \mathrm{stale}_\sigma(d) \,\} \leq \mathrm{prp}_\sigma(d)$*;*

(b) (completeness) if (R) holds and $\mathrm{rsv}_\sigma(u,d)' = \mathrm{rsv}_\sigma(u,d)$ *for every u with* $\mathrm{owed}_\sigma(u,d) \neq \emptyset$*, then u contributes a row exactly when* $\mathrm{adm}_\sigma(u,d)$*, and*

$$\mathrm{stale}_\sigma(d) \neq \emptyset \iff \mathrm{prp}_\sigma(d) \neq \mathtt{none}, \qquad \bigsqcup\{\, b : (p,u,\ell,b) \in \mathrm{stale}_\sigma(d) \,\} = \mathrm{prp}_\sigma(d).$$

Proof. Every row of u carries $b = \mathrm{pb}(u)$, and $\mathrm{prp}_\sigma(d)$ is the join of $\mathrm{pb}(u)$ over admitted units (Definition 38), so both parts reduce to comparing "u contributes a row" with $\mathrm{adm}_\sigma(u,d)$.

(a). A row means every guard passed. `bumpOf(u) == none`, `b == none` and `u.depth == 0` are Item (A1). `sources is empty` on the full source set is the first half of Item (A2), and `D in sources` is the $d \notin \mathrm{src}_\sigma(u)$ of Definition 28, read from the other end: it is the same clause as seeding `seen` with the sources. `sources is empty` after the rebinding is Item (A4). `level` is $\min\{\delta(\{P\},d) : P \in \mathtt{reaching}\} = \delta(\mathrm{owed}_\sigma(u,d),d)$, because `shortestPathsUp` gives $\delta(\{P\},d)$ for each P and a non-reaching source has distance ∞; and $\mathrm{owed}_\sigma(u,d) \subseteq \mathrm{src}_\sigma(u)$ gives $\delta(\mathrm{src}_\sigma(u),d) \leq \delta(\mathrm{owed}_\sigma(u,d),d) = \mathtt{level} \leq \mathrm{pd}(u)$, which with $d \notin \mathrm{src}_\sigma(u)$ is the second half of Item (A2) by Lemma 29. `D not in resolve(u.propagateScope)` is Item (A3); `cancelledFor` is Item (A5). Finally $\mathrm{sch}_\sigma(u)' \subseteq \mathrm{sch}_\sigma(u)$, so $\mathrm{rsv}_\sigma(u,d)'$ implies $\mathrm{rsv}_\sigma(u,d)$, which is Item (A6). So every guarded unit is admitted, and the join over a subset of the admitted units is at most the join over all of them.

(b). Conversely let $\mathrm{adm}_\sigma(u,d)$. Item (A1), Item (A2)'s first half, Item (A4), Item (A3) and Item (A5) pass the corresponding guards as above, and $d \notin \mathrm{src}_\sigma(u)$ passes `D in sources`. By (R) every owed source reaches d, so `reaching` $= \mathrm{owed}_\sigma(u,d) \neq \emptyset$, and `level` $= \delta(\mathrm{owed}_\sigma(u,d),d) = \delta(\mathrm{src}_\sigma(u),d) \leq \mathrm{pd}(u)$ by Item (A2). By the second hypothesis Item (A6) passes `resolvableBy` over the owed sources. So u contributes a row, the guarded set is the admitted set, non-emptiness is equivalent to $\mathrm{prp}_\sigma(d) \neq \mathtt{none}$, and the joins agree. □

Remark 99 (where the two formulations disagree)*.* SPEC §13.7b says the two formulations MUST agree and that disagreement is an implementation bug. As written they do not agree, and the disagreement is in the text, not in an implementation; three shapes, all on Figure 1, all with a multi-source unit and a target that got ahead of one source.

- *An owed source that does not reach the target.* Unit `feat(core,api)^: x` at C; `cli` depends on `core` only. Let `core` have delivered C to `cli` and `api` have no release carrying C. SPEC §9.2 reaches `cli` at level 1 through `core`, finds $\mathrm{owed}(u,\mathtt{cli}) = \{\mathtt{api}\}$, and bumps `cli` with provenance `api`, a package it does not depend on. The audit rebinds `sources` to {`api`}, finds `reaching` empty, and reports nothing. Definition 24 then also demands that `api` precede `cli` in the publish order whenever `cli` is tagged at h, a constraint no edge supports.
- *An owed source further away than a delivered one.* Unit `feat(core,ui)^: x` at C, target `@acme/theme`, which depends on `ui` and reaches `core` at distance 2. Let `ui` have delivered C to `@acme/theme` and `core` not. SPEC §9.2 admits at level 1 with $\mathrm{owed}(u,\mathtt{@acme/theme}) = \{\mathtt{core}\}$; the audit computes `level` $= 2 > 1$ and reports nothing.
- *Resolvability over different sets.* The same unit with `core` on `stable` and `ui` on `beta`, target `@acme/theme` on `stable`, `core` delivered and `ui` owed. SPEC §9.2 hoists `anyStable`

over the full source set and admits; the audit tests {`beta`} against a stable target and rejects, which is what SPEC §9.3a intends.

In each case the audit under-reports relative to SPEC §9.2; by Theorem 98(a) it never over-reports. The first shape is the more serious, since SPEC §9.2's answer is the questionable one: a bump owed by a package the target cannot reach, and a well-formedness constraint over a non-edge. A definition of $\mathrm{owed}(u, D)$ restricted to sources that reach D would remove the first two shapes and make (R) a theorem; the third needs SPEC §9.2 and SPEC §13.7b to agree on which source set resolvability is evaluated over. All three are flagged for the specification.

The second shape is not hypothetical: the reference engine produced it until product `main e461d7d9`, which came after the build the campaigns of Section 10 measured; a consumer that `ui` had delivered a shared commit to was released again on `core`'s account, two edges away (proof obligation F132). The specification's text permits it; the fix narrows the engine to the reaching set.

Example 100 (the audit, on both failed runs)**.** At σ_1 of the first shape of Example 39, where `core` has published and `cli` has not, the audit for $d =$ `cli` walks up from `cli`, finds `core` at $\mathtt{dist}[\mathtt{core}] = 1$, finds $\mathrm{owed}_{\sigma_1}(u, \mathtt{cli}) = \{\mathtt{core}\}$, and returns the single row

$$\mathrm{stale}_{\sigma_1}(\mathtt{cli}) = \{\, (\mathtt{core},\ u,\ 1,\ \mathsf{patch}) \,\},$$

whose join is $\mathsf{patch} = \mathrm{prp}_{\sigma_1}(\mathtt{cli})$, as Theorem 98 requires; the unit has one source, so (R) and the resolvability condition hold outright. Run the same audit for `ui`, which did publish, and it is empty: $\mathrm{dlv}_{\sigma_1}(\mathtt{core}, C, \mathtt{ui})$ empties the owed set. At σ_1 of the second shape, where `cli` proceeded and is tagged at C, the audit returns the same row for `cli`, although $C \notin W^{\circ}_{\sigma_1}(\mathtt{cli})$: this is the row the CCME 2.0.0 audit, which iterated the target's fresh window, could not produce, and it is the question an operator actually asks after a failed run (which of my packages are behind, and behind what), answered without recomputing a plan.

The clauses the proof leans on are the ones SPEC §13.7b warns an implementation drifts on, and it is worth naming what each drift produces. Excluding d from its own unit's sources: without it, a package that both changes and consumes a sibling in one unit audits as behind itself. Measuring `level` from the whole set rather than per ancestor: taking $\mathtt{dist}[p]$ for each p separately over-reports the level and so under-reports staleness, exactly in the diamond cases. Iterating the target's window rather than every retained tuple: the consumer that got ahead is invisible. And resolving `inherit` to the unit's own bump before testing it against `none`.

Proposition 101 (the tag-level screen is necessary, not sufficient)**.** *For packages P, D with baselines, say the screen of* SPEC *§13.7b* fires *when* $\mathrm{basec}_\sigma(P) \not\preceq \mathrm{basec}_\sigma(D)$*. Then:*

(a) the screen fires iff $\neg\mathrm{dlv}_\sigma(P, \mathrm{basec}_\sigma(P), D)$: it is Definition 10 with the unit's commit replaced by P's own baseline commit;

(b) if $\mathrm{adm}_\sigma(u, d)$, $P \in \mathrm{owed}_\sigma(u, d)$ and $c_u \preceq \mathrm{basec}_\sigma(P)$ (the source has released the work and its baseline reaches that release), then the screen fires for (P, d);

(c) the converse of (b) fails.

Proof. (a): $\mathrm{dlv}_\sigma(P, \mathrm{basec}_\sigma(P), D)$ asks for $t \in T_\sigma(P)$ with $\mathrm{basec}_\sigma(P) \preceq t \preceq \mathrm{basec}_\sigma(D)$; any such t gives $\mathrm{basec}_\sigma(P) \preceq \mathrm{basec}_\sigma(D)$ by transitivity, and conversely $t = \mathrm{basec}_\sigma(P)$ is a release commit of P. (b): if $\mathrm{basec}_\sigma(P) \preceq \mathrm{basec}_\sigma(d)$ then $t = \mathrm{basec}_\sigma(P)$ witnesses $\mathrm{dlv}_\sigma(P, c_u, d)$, contradicting $P \in \mathrm{owed}_\sigma(u, d)$. (c): a provider may be legitimately ahead of a consumer with nothing owed: the units between them may all declare `^none` or depth `+0`, may be excluded by `Propagate-Scope`, may reach d only beyond their declared depth, or may travel edges of

a kind not in `propagation.kinds`. Each gives a state where the screen fires and no bump is admitted. □

The screen is therefore a way to *find* candidates cheaply, by comparing two tags rather than walking a window, and never a way to decide releases; using it to decide would manufacture bumps that no commit asked for. Part (a) is why it screens: a consumer owed something has a provider release its own release does not reach. Part (b) needs the provider to have released the work; a debt from a provider that has not released the commit at all is not the screen's business, since that provider is then in the plan on its own account (Lemma 71). There is also a case the screen cannot see at all: P and d released at the same commit, in the same run, with d published first. Ancestry cannot distinguish them and no rule over tags can. That case is prevented rather than detected, by publishing in dependency order (Definition 44) and by refusing a run that would release a provider at the baseline commit of a consumer it still owes without that consumer following (Definition 24); where the consumer follows and fails, Proposition 26 is the reported remainder.

Remark 102 (historical source of the eligibility guard). CCME 1.0.0 omitted the resolvability guard, so wherever SPEC §9.3a suppressed a bump the two formulations disagreed. Proof obligation F141 preserves the counterexample; CCME 2.0.0 made the repair normative, and the current text keeps it while adding the delivery rebinding whose consequences Remark 99 records.

9 Departures of the implementation and boundaries of the text

The extended edition keeps a ledger of proof obligations open against the current specification, CCME 3.1.0-rc.2 at commit `252216ba`, and against the reference implementation at the same commit, each with the scenario that exposes it, its consequence for the results of this report, the repair or the recorded status, and the vector and test that pin it. Table 3 states each in one row; the F-numbered references open the full entries. The sources are SPEC itself, its dated design history and changelog, the engine's release notes, the integration test plan, and the test sources of the engine and its integration suite. The six obligations found in CCME 1.0.0 (the prerelease window that never discharged, the suppression lapse, the staleness dual without the resolvability test, the one-directional axis independence, the inherited channel that read a moving baseline, and the bucketing remedy's own edge) were repaired in CCME 2.0.0 and are reproduced in the extended edition as provenance with their minimal counterexamples and regression vectors (proof obligations F139 to F144).

Table 3: Current obligations, ordered by severity: the silent loss of a release first, diagnostic metadata last.

Entry	Claim	Status and consequence
Proof obligation F131	SPEC §13.3 adds an owed window per dependency edge and SPEC §19.3 refuses with `E201` a run that would release a provider at the baseline commit of a consumer it still owes. The engine implements the delivery test and the reconciliation of a proceeding consumer, and neither the owed window nor `E201`.	Open; the engine's own design history records it. A consumer that sits out the run in which its provider delivers is stranded: Theorem 74 holds in the specification and, in the engine, only while some pending window still holds the commit. The loss is silent in the plan. Pinned by vector 80d; no test exercises the run the consumer sits out.
Proof obligation F132	In the builds the campaigns ran, the bump axis credited a unit's whole source set to every dependent the walk reached, where SPEC §9.2 attributes only the sources within the unit's depth.	Fixed on the product repository's `main` at `e461d7d9`, after the measured builds and unpublished when this was written. No effect on Section 10.6: every trigger commit there scopes one package, for which both rules attribute identically. It is the (R) shape of Section 6.
Proof obligation F133	SPEC §19.3: if the consumer is published after the provider and fails, two releases sit on one commit and no later plan can compute the debt; `E201` reports it with its remedy, an explicit `Release-As`.	A boundary the specification states. Completeness (Theorem 74) reports this state rather than resolving it; the engine, which lacks `E201`, does not report it.
Proof obligation F134	SPEC §13.9's stable branch computes the graduation version from `effective(P)` over surviving tuples, while SPEC §11.5 defines it as the train target accumulated over the pending window.	A contradiction inside the text, unchanged since 2.0.0. Theorem 76 is proved for fixed version inputs; which of the two rules an implementation follows is pinned by the last row of SPEC §11.4's table.
Proof obligation F135	The engine builds one dependency set and uses it for propagation, ordering, blocking and the cycle check; the text distinguishes `propagation.kinds`, `publish.orderKinds` and `publish.blockingKinds`, and excludes a dev-only cycle from `E200`.	Open, on the safe side: the engine over-blocks and refuses dev-only cycles it should accept. The theorems assume hypothesis (K), which the engine satisfies trivially. Pinned by vectors 79 and 100a and edge cases 99a and 100.
Proof obligation F136	SPEC §19.1: a release tag is created, never replaced, except for the retry of an uncertain write that names the same commit, package and version.	The single-repository recorder implements both halves; the fleet recorder of SPEC §27 implements the rule and not the exception, so a retry after an applied-but-unacknowledged push is refused rather than recognised. Nothing incorrect is recorded. Open for the fleet path.
Proof obligation F137	SPEC §28.9 requires a stable machine-readable failure class on every listed row.	The class is attached where a distributed run loses its lock and not yet at the older recording and lock sites; the engine's design history records it as a departure to be closed. Pinned by vector 14 of SPEC §28.8.

Entry	Claim	Status and consequence
Proof obligation F138	The design history lists what the engine does not implement of SPEC §28 and where it departs: reuse of verified outputs across runs (vector 20) and the rollback profile of SPEC §26 (vector 26) are not implemented; the platform check runs before dispatch for released packages and at placement for a prepared provider; the summary shape of a prepared provider and the phase a cancelled attempt stopped in were added to the text from the implementation.	Recorded so that the conformance table of the extended edition reads against them; none affects a proved theorem.

Beside these, the extended edition records textual defects noticed in passing that change no result, and the dates and commits of the design-history entry of 22 September 2026 as observed while the product side corrected them.

10 Evaluation: the executed campaigns

This section reports what the reference implementation [5] was observed to do in the campaigns that were executed, with the binary, the environment and the counting unit of each stated separately. Five questions organise it. **RQ1** asks whether the consumer-relative admission window retains the residual obligations that a source-relative window loses. **RQ2** asks whether versions and completion records stay stable through repeated failure, repair and identity conflict, and how far the plans agree with an independent model. **RQ3** asks whether execution respects artefact availability and CI gates. **RQ4** asks what planning costs as the graph, the intent and the history grow. **RQ5** asks what the distributed execution profile of SPEC §28 measured on two real build graphs, and what it did not. Every number is transcribed from a retained file that the sentence or table note names; no campaign is pooled with another; designed deterministic matrices are reported as counts with their denominators and receive no p-values. The extended edition carries the full account of each campaign, including every failed and discarded attempt.

10.1 Evidence boundaries and units

Table 4 pins each body of evidence to its binary, environment and counting unit. The units are not interchangeable: a *history* is one generated repository planned once; an *episode* is one fixture driven through fault, retry and repair; a *state* or *transition* is an enumeration unit of the finite model; a *predicate* is one verifier check applied repeatedly to fixtures; a *release* is one `dispat release` from invocation to exit under GNU `time`. Predicate counts are implementation checks, not independent observations, and are never summed across campaigns. Every campaign through 1.10.0 runs the CCME 2.0.0 parser; the kernel and LLVM rounds run the SPEC §28 profile of the 1.11.0 release candidate and are the only evidence about it.

Table 4: Evidence boundaries. Sources: `experiments/README.md` and `experiments/results/compare-v140-v160.md` (historical); `output/experiments-v182-summary.json` (1.8.2); `paper/saner/evidence/EXPERIMENTS-1.9.0-20260909.md` and `EXPERIMENTS-1.10.0-20260909.md` (companions); `experiments/saner-2027/campaign-mechanisms-20260910/RESULTS.md` (mechanisms); the forks' `dispat/gce/PARITY-20260922*.md` and `.ci/dispat/gce/REPORT.md`, copied under `experiments/distributed-20260922/evidence/` (kernel, LLVM).

Campaign	Binary under test and environment	Counting unit and record
Historical differential, from 30 August 2026	Released `dispat 1.4.0` from the official image (release commit `5e3a2337`); repeated on 1.6.0, 1.7.0 and 1.7.2; Docker on an Apple-silicon host	History; `experiments/results/v140`, `v160`, `v170`, `v172`
1.8.2 E1 to E5, 7 September 2026	`dispat 1.8.2`, CLI commit `a7011cbc b9e88e52e7336f6fc8f1b207a8e7a311`; Docker, two CPUs and 4 GiB per container, contended host	History (E1), repetition (E2), release row (E3), episode and timing cell (E4), check (E5); `experiments/results/v182-20260907`, 753 files under its manifest
1.9.0 and 1.10.0 companions, 9 September 2026	`dispat 1.9.0`, SHA-256 `be3370c472cd bfe66fe4256d3fc1ff78ff19bdd5f041e f222187a52767fa4f35`; `dispat 1.10.0`, SHA-256 `03d1ba5fbaef35b69582e9489f 40b5438f81ae6926e02ea6d02716b5b73d9 2fd`; Docker Desktop Linux ARM64, two CPUs, 4 GiB, private registries only	History, episode, execution, CI repetition, predicate; `experiments/saner-2027/campaign-next-release-prepared/runs` (one failed and one completed attempt) and `campaign-1.10.0-20260909` (9,157 archived files)
Mechanism matrix, 9 September 2026	The 1.10.0 binary and runner image; network disabled	Episode, state, transition; `experiments/saner-2027/campaign-mechanisms-20260910`, 16,464 fixture files archived
Kernel farm rounds, 22 September 2026	Round 1: a development build, SHA-256 `a7172a7feb21e27ce882050120b9e04d f1e6d4b4ee5f3341ab8a99ba155ba70a`; round 2: the feature branch at `f0c9e959`, static linux/amd64, SHA-256 `63441cc 66f68d07b5efbf9a42edd5559304d153 d48b82b1dfb037349e20606ed`; Google Compute Engine [19], `c3-standard-22` and `t2d-standard-8`, us-central1	Release (GNU `time`), per-cell script seconds; `dispat/gce` of the fork at `c699bf3d`, copied under `evidence/kernel-gce`
LLVM pair, 22 September 2026	Attempts 1 to 8: `f0c9e959`; attempt 9: `1.11.0-rc.4+main.272772cb` as printed in its release log; Compute Engine, `e2-standard-4` orchestrator, two `n2-custom-14-49152` workers, us-central1-a	Release; the fork's `.ci/dispat/gce/REPORT.md` and the logs under `evidence/llvm-gce/logs`

10.2 RQ1: the consumer-window mechanism

The consumer-relative window of Definition 9 is the mechanism behind **G2**. The mechanism campaign tests it two ways (`campaign-mechanisms-20260910/PROTOCOL.md`). The *finite projection* enumerates every DAG consistent with one fixed order of four named vertices, each of four single-intent sources, every completion set reachable by recording an enabled package, and every subset of the remaining packages designated as failed; it checks exact residual obligations, no widening, no double ledger delivery and strict decrease exactly on progress, then applies the source-window mutant, which discards a unit when its originating package

completes. From the `executor.model` object of `summary.json`: 64 graphs, 256 source instances, 772 reachable completion states, 2,270 failure-subset transitions of which 1,054 made progress, 0 projection violations, and 260 states in which the mutant lost obligations. States and transitions are enumeration units, not samples, and the projection is a stable-channel, one-intent model of four vertices, not a mechanised proof.

The *binary-derived counterfactual* takes each episode's original obligation set and its observed completion tags and asks what the source-window rule would have discarded; it runs no modified binary and compares no competitor. Table 5 transcribes the strata.

Table 5: Source-window counterfactual by stratum: the `episodes`, `source_mutant_losing_episodes` and `source_mutant_lost_obligations` fields of each `strata` entry in `experiments/saner-2027/campaign-mechanisms-20260910/summary.json`. Units are episodes and obligations.

Stratum	Value	Episodes	Losing episodes	Lost obligations
Class	fault matrix	120	84	111
	no-fault control	10	0	0
	foreign identity	10	10	10
Boundary	build	40	28	41
	before publication	40	28	35
	after artefact write	40	28	35
Channel	stable	70	47	61
	beta	70	47	60
Graph	chain	28	20	38
	diamond	28	20	25
	fan-out	28	20	20
	two disconnected pairs	28	14	14
	chain with shortcuts	28	20	24
All		140	94	121

*Answer to **RQ1**.* In every state the projection reached and in every episode the binary completed, the consumer-relative window retained exactly the residual set; a source-relative window would have lost obligations in 260 of the 772 states and in 84 of the 120 fault episodes (111 obligations, 70%). The ten no-fault controls lose nothing; the ten foreign-identity cases lose one obligation each and test a different intervention.

10.3 RQ2: retry and identity stability

The mechanism matrix. Five four-package topologies (chain, diamond, fan-out, two disconnected pairs, chain with shortcuts), stable and beta baselines, four fault targets and three injection boundaries give 120 fault episodes; ten no-fault controls and ten foreign-identity cases bring it to 140. Each fault episode executes the same fault twice, repairs it and runs a final no-op, in an order fixed by seed 20260910, with build and publish concurrency one. The run took 113.653827 s on 9 September 2026 and passed 140 of 140 episodes (`summary.json`, `execution` and `executor`). Table 6 gives the five checks; per stratum, 40 of 40 episodes passed at each boundary, 70 of 70 in each channel and 28 of 28 on each graph. An independently implemented offline checker reconstructed every plan from the raw command output and re-enumerated the finite model with no discrepancy, and a negative check on a corrupted copy failed as intended (`RESULTS.md`). The after-write injector lets the publisher write the artefact durably and then return a nonzero exit; it is not a crash. The identity check is the supplied local publisher's, not a registry's.

Table 6: Mechanism checks of the 140-episode matrix, from the `mechanism_checks` object of `experiments/saner-2027/campaign-mechanisms-20260910/summary.json`. The 130 denominator is the 120 fault episodes plus the ten foreign-identity cases.

Check	Passed / denominator
Repeated fault preserved the residual plan and tags	130 / 130
Successful repair left an empty plan	130 / 130
Exactly one durable artefact write per obligation	140 / 140
Foreign artefact identity conflict was refused	10 / 10
Final no-op left the plan empty and tags unchanged	140 / 140

Reference agreement and recovery in the companion campaigns. The 1.10.0 campaign planned 300 generated histories, seeds 90001 to 90300, against the restricted core oracle: 300 accepted matching rows, no rejection, invalid execution or difference (`campaign-1.10.0-20260909/summary.json`, `core`); the corrected 1.9.0 campaign reports the same 300 of 300 on the same seeds. Agreement on a fixed sample with the historical restricted model is not a universal conformance result. The recovery stage drives 20 episodes over graphs of 20 and 100 packages with failure probabilities .1 and .3: in 1.10.0 all 20 converged within four runs (nine episodes in two runs, five in three, six in four) over 251 raw command records; in 1.9.0 within four runs over 248 records (nine, six, five). The two are not pooled; the difference is one extra attempt in the episode of size 100, probability .3, seed 100, whose later seeded deny sets diverged, and the traces do not attribute it to the 1.10.0 feature.

The 1.8.2 full-profile campaign. On the released 1.8.2 binary, the core shards agreed 1,000 of 1,000 and 500 of 500 with no invalid execution. The full profile deliberately generates regressive prereleases: of 1,500 instances, 503 were refused with a validated `E195` and 997 accepted; 546 of the accepted matched the historical oracle and four refused dossiers had no row difference, so 550 comparisons were diagnostic-clean; the 1,914 divergent package rows were classified PG1 862, PG2 375, PG2x 14 and PG3? 663 (`output/experiments-v182-summary.json`, `E1`). A refused diagnostic plan is not a release plan, the oracle implements the historical CCME 1 model, and the PG3? label is heuristic: these counts are taxonomy against that model, not CCME 2.0.0 conformance. The first 1.8.2 attempt, whose strict exit checker misclassified the expected refusals, is retained at `experiments/results/v182-20260907-attempt1`; the corrected campaign reran every seed. Appendix B vectors 133 to 138 passed 6 of 6 on the same binary (`CCME2`).

Historical differential, orphan, push, competitor and replay observations. On the released 1.4.0 binary the core profile agreed 1,000 of 1,000 and 500 of 500; the full profile's divergent rows fell into the four predicted classes and none outside them (PG1 409, PG2 240, PG2x 58, PG3 390 on the first shard), and the 1.6.0, 1.7.0 and 1.7.2 campaigns reproduce the per-seed rows on all 3,000 instances (`experiments/README.md`, `compare-*.md`); these validate the CCME 1.0.0 model against those binaries, not CCME 2.0.0 admission. E2 drove a six-package npm fixture against a Verdaccio registry behind a failing proxy: both repetitions recorded the failed publish untagged, a same-version recovery and a final plan releasing nothing. E5's three fresh cells passed 8 checks for the orphan scenario, 12 for a clean concurrent push and 14 for a conflicting concurrent push, on selected interleavings rather than a sweep (`E2`, `E5`). The competitor phases of 30 August 2026 (Lerna 10.0.1, Nx 23.1.2, Turborepo 2.10.12 with Changesets 3.0.1, release-please 17.11.2) are observations of pinned configurations: Lerna and Nx tagged before publish, Changesets after and by registry query, Turborepo neither versions nor publishes. E3a replayed five releases each of two public repositories under

reconstructed layouts: 36 of 42 released package-versions exact for `google-cloud-node` and 7 of 64 for `conventional-changelog` (`experiments/results/e3a/gcn.json`, `cc.json`); neither project used the modelled intent language, and the experiment shows the limit of policy transfer, not that those releases followed CCME.

*Answer to **RQ2**.* Across the designed factors of the 140-episode matrix, recorded progress survived repeated failure without version drift, repair emptied the plan, and the identity check refused every foreign artefact. The restricted oracle agreed on all 300 histories of each companion campaign and on all 1,500 core instances of the 1.8.2 campaign; the divergent full-profile rows are retained and classified, and mark where the restricted models stop rather than where the implementation fails.

10.4 RQ3: availability boundaries and CI

The companion campaigns execute two registry topologies and one CI fixture: an npm package feeding a Docker base feeding a Docker application, and its four-package extension `assets` (npm), `ui` (npm), `base` (Docker), `app` (Docker), where `ui` reads its predecessor's local `dist` output. Each topology repeats five configurations six times in a frozen seeded order: mixed local-output scheduling, fully serialised controls, a missing barrier, an injected base-publication failure with repair, and an injected app-build failure with repair (Table 7).

Table 7: The 1.10.0 companion campaign, from the `mixed` and `integration` objects of `experiments/saner-2027/campaign-1.10.0-20260909/summary.json`. Predicates are repeated verifier checks, not samples.

Topology	Executions	Initial successes	Designed failures	Repairs	No-op reruns	Predicates
Three-package	30	12	18	12	24	522
Four-package	30	12	18	12	24	798
CI fixture	3 repetitions					81

Per topology the 18 designed failures are six of each negative configuration; all 12 repairable executions completed at unchanged planned versions; the 24 completed cases performed no build or publication on their final unchanged run. In the three-package mixed fixtures the base build overlapped npm publication in all six observations, by 0.110716 to 0.151483 s; in the four-package fixtures the `ui` build overlapped the delayed `assets` publication by 0.070128 to 0.073725 s and the base build completed before the `ui` publication began in all six cases, an admissible schedule (`EXPERIMENTS-1.10.0-20260909.md`). The three CI repetitions each cover baseline publication, a specification change, a consumer change, an injected failing test, a missing required test, repair and an unchanged run; both negatives blocked the release stages and the failing-test attempts wrote no release tag. The corrected 1.9.0 campaign has the same counts and the same 522, 798 and 81 predicates. Its first attempt is preserved as failed: the original four-node oracle demanded strict interval overlap, the traces showed the valid schedule in which the base build completed before `ui` publication began, and the predicate was corrected before the complete rerun, whose protocol 1.10.0 then reused byte for byte (manifest SHA-256 `a3366e2fbdc590be7e8f6268bc7390146669cf7a0395837c2531a0af5554dcaa`). A separately frozen diagnostics addendum passed 16 of 16 cases outside the 14-stage campaign.

*Answer to **RQ3**.* In these fixtures, on one Linux ARM64 environment, a configured barrier distinguishes local build availability from required publication, and both a failed and an absent required test prevent release; the campaign measures neither concurrency at scale nor onboarding effort.

10.5 RQ4: planning cost

The E4 timing cells plan generated workspaces of P packages with U intent units in three graph shapes, with an optional padding history of commits that touch no package (Table 8).

Table 8: Planning cost, 1.8.2 campaign: the eight rows of `experiments/results/v182-20260907/e4/timing.csv`, verbatim. Seconds are the median, minimum and maximum of the cell's repetitions; MiB is the maximum resident set size; two CPUs and 4 GiB on a contended Apple-silicon Docker host.

P	Shape	U	Padding	Edges	Median s	Min s	Max s	MiB
100	layered	200	0	185	0.017	0.017	0.017	15
1,000	layered	200	0	2,002	0.250	0.235	0.252	21
10,000	layered	200	0	19,919	19.272	18.992	20.000	92
1,000	chain	200	0	999	0.254	0.241	0.258	26
10,000	chain	200	0	9,999	19.354	18.933	19.645	118
1,000	star	200	0	999	0.244	0.237	0.247	19
1,000	layered	1,000	0	2,002	0.290	0.280	0.302	26
1,000	layered	200	5,000	2,002	0.664	0.640	0.665	30

The medians rise from 0.017 s at 100 layered packages to 19.272 s at 10,000; five-fold intent at 1,000 packages moves the median from 0.250 to 0.290 s and 5,000 padding commits to 0.664 s, the history term of the cost analysis showing through. The earlier campaigns measured the same headline cells under a different harness at 0.6, 2.8 to 6.7 and 110 to 149 s; those figures are not comparable, the runner having been rewritten before the 1.8.x campaigns, and neither set is a cross-version benchmark. The E4 recovery cells converged in 24 of 24 episodes within four runs.

*Answer to **RQ4**.* On the capped host the observed cost grows sharply between the 1,000- and 10,000-package cells and with history length; the design isolates neither an asymptotic bound nor an end-to-end figure.

10.6 RQ5: distributed execution

The execution profile of SPEC §28 places build and publish stages on worker nodes that share a Git repository with the orchestrator as their only transport. Two build graphs were measured on 22 September 2026, both on Google Compute Engine: the Linux kernel image-build farm of kerneltests.org [15], whose cells are independent, and the LLVM monorepo, whose cells consume one another's outputs. The specification's own record of both measurements is the entry "2026-09-22: Distributed execution implemented" of `DESIGN-HISTORY.md`, which this subsection follows. The claim the measurements support is narrow and is stated once: on these fixtures, once each, a distributed release took no longer than the same recipe on one node; no speedup of the profile or of the engine is claimed beyond that. The raw observations are copied unedited into `experiments/distributed-20260922/evidence/`, whose `MANIFEST.json` records every file's origin and SHA-256 and whose `CROSS-CHECK.json` traces every headline figure below to a raw file (43 checks, 41 comparable, all agreeing). Each fork carries its own acceptance script: the kernel fork's `dispat/compare.py` accepts a run only if its cases, revision, compiler and assembler identities match the public reference, every cell succeeded, and the elapsed time is no greater than the reference; the LLVM fork's `.ci/dispat/gce/compare.py` compares the install trees file by file with the prefix masked.

10.6.1 The kernel farm

The public reference for each group is the build step of the upstream builder for kernel v7.3-rc4 at `93f51579e7df248780214094418f205253383cc5`: alpha 623 s, hexagon 111 s and m68k_nommu 128 s, with GCC 16.2.0 over GNU assembler 2.47.20260726 from the kernel.org crosstool builds [18] and Ubuntu clang 18.1.3 for hexagon (`evidence/kernel-gce/round1-codex/baseline.json`). These are build-step durations on the farm's own machines while sharing them with another build, on other hardware: they bound acceptance and enter no other comparison. A first run on 22 September established the coordination profile and two engine defects and is not parity evidence (wrong compiler, results not retained). Round 1 then installed the reference toolchains on a `c3-standard-22` and a `t2d-standard-8` worker and measured a grouped control and four candidates, each once; every candidate failed the gate on elapsed time alone (split hexagon 170.07 s, split m68k_nommu 159.03 s, batched hexagon 165.30 s, batched m68k_nommu 196.23 s; the control 1,797.75 s with alpha's script alone at 1,324 s against 623), with cases, toolchains and revision accepted (Table F11; the curated archive of 145 files, SHA-256 `2fcc02ec...`, is under `evidence/kernel-gce/round1-codex/`).

Round 2 calibrated the farm's script bodies outside dispat on prepared source trees (`round2/calibration/cal-c3.txt`): the three hexagon configurations started together at 11 jobs each finished after 120.12 s, the five m68k_nommu configurations at 7 jobs each after 128.17 s, both at or above their references before any release overhead. It then pinned each group's costliest cells to the `c3-standard-22` that starts the release and gave the rest to the `t2d-standard-8` worker, leased a prepared source tree per build slot instead of a per-cell worktree, and recorded the worker's cell on the orchestrator. Table 9 transcribes every timed release of the round, including the rejected first candidate and a rerun that planned nothing.

`compare.py` accepted the exact case lines, compiler, assembler and revision in all seven releases that built cells and rejected three on elapsed time alone; every cell reported `passed`. The gate was met twice for hexagon after one rejected candidate at 111.47 s (a 12.0 s lease wait on each of its three cells, before the leased-tree change), and twice for m68k_nommu with none. The same-hardware pairs are the single-node rows: each group run once more on one `c3-standard-22` with no worker and nothing else changed, rejected on elapsed time alone at 122.62 and 138.01 s. Two features of those runs are stated rather than hidden: a release that names no worker is not bounded by `execution.concurrency` and pays no preflight, so its figure is the best the machine can do with the recipe; and the lease check dominated the m68k_nommu run, two cells recording 58 and 66 s of `source_lease_seconds` against build scripts of 73 and 72 s (`solo-c3/results/*/result.json`), so 138.01 s is pessimistic by some part of that cost. The calibration outside dispat is the corroboration. alpha was not run: its `allmodconfig` is one cell that cannot be split, with a lower bound of about 1,198 s against a budget of about 610 s on the largest machine the project's 32-vCPU quota allows; a group the experiment could not measure, not one it measured and failed. The engine has two placement classes and no per-worker pinning, which sufficed because each group needed exactly two machines; the worker sat idle 59 and 49 s in the m68k_nommu trials while the orchestrator finished four cells. Fixed overhead per release was 3 to 8 s of preflight; the round's five VMs lived 10 to 22 minutes each.

10.6.2 The LLVM pair

The fork `yohimik/llvm-project`, branch `dispat-gce`, LLVM `main` at `a9f4d3ff`, builds with `LLVM_FAST=1` (assertions, examples, docs and benchmarks off), `LLVM_SKIP_CHECKS=1`, the X86 target, 14 build jobs and 2 link jobs, on an `e2-standard-4` orchestrator and two `n2-custom-14-49152` workers (7 cores each), Ubuntu 24.04.5, g++ 13.3.0, cmake 3.28.3, ninja 1.11.1.

Table 9: Kernel round 2, every timed release. Elapsed is the GNU `time` record (`par-c3/*.time`, `solo-c3/*.time`, also `*.elapsed`); script and lease seconds per cell and the `compare.py` verdict with its ratio (reference over elapsed) are from `verdicts.txt`; all under `dispat/gce/round2` of the fork and under `experiments/distributed-20260922/evidence/kernel-gce/fork-codex-kernel-parity-c699bf3d/dispat/gce/round2`. Each row is one release.

Run	Cells: script s (lease s)	Elapsed s	Ref. s	Verdict, ratio
hexagon, first candidate	c3: tinyconfig 60 (12.0), defconfig 84 (12.0); w1: allnoconfig 80 (12.0)	111.47	111	reject, elapsed only; 0.9958
hexagon, rerun of the unchanged configuration	no cell built: every package reported `unchanged`	3.83	111	not compared; planned nothing
hexagon, trial 1	c3: tinyconfig 70 (0.0), defconfig 85 (0.0); w1: allnoconfig 82 (0.0)	95.89	111	accept; 1.1576
hexagon, trial 2	c3: tinyconfig 71 (0.0), defconfig 84 (0.0); w1: allnoconfig 81 (0.0)	92.22	111	accept; 1.2036
m68k_nommu, trial 1	c3: m5249evb 95 (1.0), m5272c3 96 (1.0), m5407c3 96 (14.0), m5307c3 97 (15.0); w1: m5475evb 50 (0.0)	115.61	128	accept; 1.1072
m68k_nommu, trial 2	c3: m5249evb 105 (0.0), m5407c3 105 (0.0), m5307c3 106 (0.0), m5272c3 107 (0.0); w1: m5475evb 51 (0.0)	115.48	128	accept; 1.1084
hexagon, one node alone	solo: tinyconfig 91 (13.0), allnoconfig 105 (13.0), defconfig 110 (12.0)	122.62	111	reject, elapsed only; 0.9052
m68k_nommu, one node alone	solo: m5475evb 56 (0.0), m5407c3 91 (0.0), m5307c3 92 (0.0), m5272c3 73 (58.0), m5249evb 72 (66.0)	138.01	128	reject, elapsed only; 0.9275

The recipe's identity record embeds the toolchain versions, the source revision and its own hash into every receipt, and a consumer refuses a provider whose receipt does not match the installed tree. Nine attempts were made; six have a retained orchestrator log and five a GNU `time` record (`evidence/llvm-gce/logs/`, mapped to attempts in `MANIFEST.json`; Table F13). Six were discarded or failed for reasons recorded with them: the recipe's fingerprint hashed umask-dependent modes; an orchestrator without a C++ compiler; a baseline clone without the `v24.0.0` tags; a fixture change; and a dispat defect, captured outputs passed through the checkout's `.gitattributes` text conversion, which rewrote CRLF pairs inside three libraries and failed the bundle in attempts 4 and 7, fixed on `main` at `9f649f88`. A second engine defect, a fixed 30 s report bound that killed the 1.4 GB result push, was fixed at `f0c9e959` before the pair. The accepted pair is attempts 8 and 9.

The single-heavy fixture of attempts 4 and 5 (`llvm`, then `lld` and `polly` consuming it) is the negative case SPEC §28.7 predicts: on one node the three were published at 20:16 (whole release 36:09.09 with a 15:53 xz bundle); distributed, at 26:47.69, a wall time that ends in exit 1 on the bundle. One package holding nine tenths of the work cannot pay for moving its 1.4 GB install tree to consumers that build in two minutes. The matrix fixture adds `llvm-aarch64`, the same sources built for AArch64 as a package that depends on nothing, so that one node must serialise two heavy builds and a pool need not, and sets an uncompressed bundle on both sides (Table 10).

`llvm`'s declared outputs, 1.4 GB in 2,647 files, were pushed once and fetched by two consumers

Table 10: The LLVM matrix pair, attempts 8 and 9. Per-package figures are the results table of `.ci/dispat/gce/REPORT.md`; the whole-release wall times are the `Elapsed (wall clock) time` lines of `release-local3.time` and `release-dist6.time` under `experiments/distributed-20260922/evidence/llvm-gce/logs/`, whose `done` lines record 5 published and 0 failed on both sides. One repetition per side.

	Baseline, `w2` alone	Distributed, `w1` + `w2` + `lb-orch`
`llvm` (X86)	38:38, sharing the node	22:08 on `w1`, including the 1.4 GB result push
`llvm-aarch64`	38:51, sharing the node	23:22 on `w2`, including its result push
`lld` + `polly`	2:16	2:13 on the two workers, after `llvm`'s outputs reached them
bundle (uncompressed tar)	0:18	0:32 on `lb-orch`
whole release, GNU `time`	41:12.02, exit 0	27:48.94, exit 0

and the orchestrator; `lld` (420 MB) and `polly` (22 MB) consumed them on nodes that never built `llvm`, which is the mechanism SPEC §28.7 names as the profile's required gain. The bundle verified every provider's receipt against the installed tree, so the transport is byte-faithful; `compare.py` then finds the two sides differ only in what the recipe embeds about its checkout (the compiled-in VCS revision and the build-id derived from it, the repository string, RPATH and `__FILE__` paths), and `libLLVMCore.a`, which embeds none of these, is byte-identical; byte identity throughout needs `LLVM_APPEND_VC_REV` off, a file prefix map and a relative RPATH, which this recipe does not set. Coordination was cheap beside the builds: probe 1 to 4 s, claim 3 to 6 s warm, a result noticed within 5 s; a cold worker's first claim comes minutes later because its poll fetches the snapshot branch first, about 10 min for the 3.6 GB history.

Binaries and caveats. Attempt 9 ran `1.11.0-rc.4+main.272772cb`, built in a checkout with uncommitted documentation edits, so that Go stamped it `vcs.modified=true`; the deployed binary is recorded by its SHA-256 prefix `feba0210d13c90fa`, and the reproduction kit's clean rebuild of the same commit gives `46bc2cf821dd57479c27bf2a4b7eee63b69e01db6f18fbd6e663ad66e6812ba8`, differing only by that stamp; `f0c9e959` rebuilds byte-identically to `63441cc66f68d07b5efbf9a42edd5559304d153d48b82b1dfb037349e20606ed`. Both measured builds credited a unit's whole source set to every reached dependent until product `main e461d7d9`; the fixtures carry single-source units only, which both rules attribute identically (proof obligation F132). The qualifications the specification's own entry states are repeated without softening: one repetition per side on both graphs, so variability is unmeasured; the kernel's second node is a slower machine of another family, so each kernel pair is one node against that node plus a second, slower one; the kernel groups have only independent cells to distribute; the LLVM matrix's two heavy builds are one project for two targets; the LLVM distributed side had two build nodes against one; and the single-node m68k_nommu figure includes 58 to 66 s of lease contention. The entry dates the LLVM pair 2026-09-23 while every retained log is stamped 2026-09-22 UTC, and names the orchestrator-side commit `6c1a5fcd` where the fork's is `d8fb8408`; both are recorded as observed, the driver record `llvm-gce/drivers/pair4.out` showing the second to be the same change applied on the orchestrator as a patch.

*Answer to **RQ5**.* On the kernel fixture the distributed release met the public reference gate twice for each of two groups (hexagon 95.89 and 92.22 s against 111, after one rejected candidate; m68k_nommu 115.61 and 115.48 s against 128) where the same recipe on the same single node did not (122.62 and 138.01 s), each single-node run once; alpha was outside the project's capacity. On the LLVM fixture the distributed release of a five-package matrix

took 27:48.94 against 41:12.02 on one node, produced the same five products under the recipe's receipts, and the negative case predicted by SPEC §28.7 was observed at 26:47.69 against 20:16. On these fixtures, once each, distributed took no longer than single node; nothing more is claimed.

10.7 Threats to validity

The mechanism matrix is a designed, deterministic set of 140 episodes with no population inference; its after-write fault is an adapter response, not a crash; the projection is four vertices, one intent and the stable channel; the counterfactual runs no modified binary; the runner image cannot be rebuilt bit for bit on a fresh machine because `apk add` consults an external repository. The companion campaigns' oracle is the restricted core profile; their predicate counts are not independent observations; their delays expose availability, not throughput; their environment is one Linux ARM64 Docker host and validates nothing about native Windows, which no campaign measures. The E1 oracle is the CCME 1.0.0 model, so its divergence classes are taxonomy against that model; the E3a replay reconstructs layouts the projects never declared; planning-cost figures are host-sensitive and no cross-version benchmark. The kernel rounds are one pair of machine shapes, one region, one kernel revision, two trials per accepted topology and one single-node run per group, gated against public build-step times from other hardware; alpha, every QEMU group and the 57-group matrix are unmeasured. The LLVM pair is one repetition per side with two build nodes against one, the same sources for two targets, a recipe without the check targets, six of nine attempts failed or discarded, a deployed binary recorded by hash prefix only, and raw logs copied from the measuring session's scratchpad into the evidence tree rather than kept in the fork.

10.8 Reproduction

The Docker and Terraform kit for the distributed measurements is at `experiments/distributed-20260922/` (README, `pins.json`, node images, Terraform roots, runbooks, a local Compose check), beside the `evidence/` directory. Verified on 22 September 2026 on an Apple silicon workstation: the experiment binaries rebuild byte for byte from their commits in a digest-pinned `golang:1.26.5-alpine` to the hashes above; the four round-2 configurations are regenerated by the fork's generator byte for byte apart from the mailbox path; both Terraform roots validate and plan against the original projects (thirteen resources each) but `terraform apply` was not run, so no cloud pair was re-measured; and the pipeline ran end to end on the workstation with two containers of the kernel node image, the fork's `compare.py` accepting the run (three cases, revision `93f51579`, 63.47 s, a laptop figure that says nothing about parity; `runs/local-kernel-20260922T184954Z/verdict.json`). A reproducibility audit of every older campaign, with rerun commands and status per family, is `experiments/REPRODUCIBILITY.md`.

A Notation

Symbol	Meaning	Defined
$\mathcal{K}$, h	commits; the fixed HEAD	Definition 1
reach(c), $c' \preceq c$	ancestors-or-self of c; c' is one of them	Definition 1
$\mathcal{P}$, E, dep(p)	packages; dependency edges; dependents of p	Definition 3
K_{prop}, K_{order}, K_{block}	the propagation, ordering and blocking edge kinds	Definition 3

Symbol	Meaning	Defined
$\delta(S, d)$	least edge count from a member of S to d	Definition 5
$R(S, \ell)$	the traversal: $\{d \notin S : \delta(S, d) \leq \ell\}$	Definition 28
$\mathbb{B}$, $\sqcup$	none $<$ patch $<$ minor $<$ major; their join	Definition 6
$\mathcal{V}$, $\mathcal{C}h$, stable	versions; channels; the stable channel	Definition 7
$\mathrm{ch}(v)$, $\mathrm{core}(v)$, $\mathrm{cnt}(v)$	channel, core triple, prerelease counter of v	Definition 7
$\mathrm{apply}(v, b)$	v with bump b applied	Definition 7
σ, $\sigma \leq \sigma'$	engine state (the reachable tags); inclusion	Definition 8
$T_\sigma(p)$	release commits of p; $h \in T_\sigma(p)$: tagged at h	Definition 8
$\mathrm{base}_\sigma(p)$, $\mathrm{basec}_\sigma(p)$	baseline version of p; its commit	Definition 8
$\mathrm{sbase}_\sigma(p)$, $\mathrm{scom}_\sigma(p)$	stable baseline of p; its commit	Definition 8
$W_\sigma(p)$	train window of p	Definition 9
$W^\circ_\sigma(p)$	undelivered (fresh-admission) window	Definition 9
$\mathrm{dlv}_\sigma(P, C, D)$	P has delivered C to D	Definition 10
$W^{\mathrm{ow}}_\sigma(P, D)$	owed window: what P has not delivered to D	Definition 11
$\mathcal{U}$, c_u, $\mathrm{scope}(u)$	units; a unit's commit; its resolved scope-set	Definition 16
$\mathrm{bump}(u)$	the bump u's type and ! map to	Definition 16
$\mathrm{pb}(u)$, $\mathrm{pd}(u)$, $\mathrm{ps}(u)$	bump-axis directives of u	Definition 16
$\mathrm{pc}(u)$, $\mathrm{pcd}(u)$, $\mathrm{pcs}(u)$	channel-axis directives of u	Definition 16
$\mathrm{canc}(c, p)$, $\mathrm{held}(p)$	the two suppressors	Definition 19
$\mathrm{disch}_\sigma(p, c)$	p has already released the work in c	Definition 19
$\mathrm{src}_\sigma(u)$	source packages of u	Definition 19
$\mathrm{owed}_\sigma(u, D)$	sources of u that have not delivered c_u to D	Definition 19
$\mathrm{owes}_\sigma(P, D)$	P is owed to D by some admitted unit	Definition 19
$\mathrm{sch}_\sigma(u)$	channels of u's sources	Definition 35
$\mathrm{rsv}_\sigma(u, d)$	d can resolve what u's sources release	Definition 35
$\mathrm{adm}_\sigma(u, d)$	u propagates to d	Definition 37
$\mathrm{chan}_\sigma(p)$	the channel p is released on	Definition 34
$\mathrm{dir}_\sigma(p)$, $\mathrm{prp}_\sigma(p)$, $\mathrm{eff}_\sigma(p)$	direct, propagated, effective bump	Definition 38
$\Pi(\sigma)$, $\nu_\sigma(p)$	the plan; p's planned version	Definition 21
$\mathrm{own}_\sigma(p, X)$	p has a cause of its own relative to X	Definition 23
$\mathrm{wf}(\sigma)$	no provider planned at the baseline commit of a consumer it owes without that consumer following (E201)	Definition 24
$\mathrm{run}(\sigma, F)$, $S(\sigma, F)$, $B(\sigma, F)$	one run, failing on F; what it publishes; what it blocks	Definition 25
$\mathrm{stale}_\sigma(d)$	the upward audit of SPEC §13.7b	Definition 97
(H1), (H2), (R), (K)	standing hypotheses	Section 6

Diagnostic codes (E200, E201, W208, and the rest) are those of SPEC §16 and are used here with their registry meanings.

B The conference abstract, read against this report

The companion conference paper's abstract was registered on 22 September 2026 and is fixed. This appendix takes it sentence by sentence and says where this report substantiates each statement, with the precision the retained evidence allows, so that no sentence of the abstract is read further than the report supports. Statements about product features that this report neither models nor measures are marked as such and located in the product repository at commit 252216ba, the snapshot this report transcribes. Two facts postdate the abstract and

appear only here: the reached-set attribution defect of the measured builds (proof obligation F132) and the specification entry's wrong date for the LLVM pair (Section 10.6.2).

Table 12: Each sentence of the conference abstract, where the report substantiates it, and the precise statement.

Abstract	Where	Precise statement and caveat
"An npm engine feeds a Docker base; another image needs that published base and an npm SDK. If the final image fails, its release must remain pending after its providers succeed."	Section 2 (Definition 9), Theorem 74; Section 10.4, Table 7	The consumer-relative window keeps a contribution admissible until the consumer releases it (Theorem 74). The three- and four-package registry topologies of the companion campaigns exercise the npm-to-Docker chain: per topology, 18 designed failures and 12 repairs at unchanged planned versions, on one Linux ARM64 Docker host. Which consumer waits for a publication is the declared edge's matter: a Docker consumer built in an isolated registry-based builder needs the published base, a consumer that reads local output does not (Section 5.1).
"We present a Git-based protocol that tags completed releases and resumes unfinished work through the same release command."	Section 4; Theorem 94; Section 10.3	Recorded legs are annotated tags and recovery is re-execution of the same command (Theorem 94). Observed: 20 of 20 recovery episodes converged within four runs in each companion campaign, and 130 of 130 repeated-fault rows of the mechanism matrix preserved plan and tags (Table 6). The theorems concern the model under the retry invariant; the implementation's departures are the ledger of Section F10.1.
"Folders define packages and shell commands define stages across languages, single-package repositories, monorepos and polyrepos."	Section F5.3; product documentation	The report models the release computation and transcribes the three history modes of SPEC §27. The configuration language (folders as packages, stages as commands) is the product's, documented under `packages/docs/docs/configuration/` at `252216ba`, and is not modelled here. No campaign of Section 10 evaluates a polyrepo or discovered-history release; those semantics are transcribed, not evaluated, and the theorems of Section 6 are stated for a single history.
"Commits select releases, holds, prereleases, dependent updates and tests; an agent skill supports diagnostics and automation."	Section 2, Section 3; Section 10.4; product `specs/agent-guide/README.md`	Units, scopes, depth, channels and holds are the model of Sections 2 and 3; test selection by commit is the CI fixture of Section 10.4 (three repetitions, 81 predicates). The agent guide (version 1.11.0-rc.3 at `252216ba`) is a product document for coding agents; this report neither models it nor measures its effect, and no campaign measures agent effectiveness.

Abstract	Where	Precise statement and caveat
"Shared Windows/Linux configuration, containerized tools and extensible manifest readers let provisioned CI reuse its workflow as packages are added."	product documentation only	Outside this report's model and campaigns. Shared platform configuration, containerised tool execution and the manifest scanner and writer are product features documented at `252216ba` (`packages/docs/docs/configuration/`, `packages/docs/docs/cli/scanner.md`, `cli/writer.md`). No campaign measures onboarding effort, and native Windows execution is unmeasured everywhere (Section 10.7).
"Terraform integration reconstructs temporary state from known resources before dependent deployments."	Section 4, the recorded-success remark; product `packages/docs/docs/examples/terraform.md`	The protocol section states what a Terraform-backed package's tag records and does not: that a configuration version completed; it is not Terraform resource state and does not prove that every live resource was imported. No campaign of Section 10 exercises the integration.
"Identical inputs produce identical, cacheable plans; the distributed profile shares configuration and transports sources and results through Git."	Section 7; Section F9; Section F5.2; Section 10.6	Determinism and idempotency are proved for the model (Section 7); the cache-reuse and optimisation claims of SPEC §13.11 and SPEC §20.7 are restated with their qualifications in Section F9; placement invariance of the plan is the invariance Section F5.2 states for SPEC §28. Two releases through Git-mailbox workers were measured, once per side (Section 10.6). The measured builds carried the attribution defect of proof obligation F132, which does not affect single-source units.
"Our evaluated single-history model builds on sagas and version control; retry assumptions preserve versions and avoid repeating recorded releases."	Section 1; Theorems 76, 79, 81 and 85	Version stability, no double ledger delivery, no widening and convergence are theorems of the model under the explicit retry invariant (Section 6); they say nothing unconditional about external effects, and convergence counts progress-making runs only. Sagas are credited in Section 1.
"All 120 controlled fault cases recovered after repair."	Section 10.3, Table 6	The 120 fault episodes of the mechanism matrix (five graphs, two channels, four targets, three boundaries), each executed twice under the fault, repaired and rerun: 130 of 130 repairs left an empty plan (120 fault plus 10 foreign-identity cases) and 140 of 140 final no-ops left plan and tags unchanged. A designed deterministic matrix, not a sample; the after-write fault is an adapter response, not a crash.
"Discarding consumer work at provider completion would instead lose releases in seven out of ten cases (analytical comparison, not a tool benchmark)."	Section 10.2, Table 5	84 of the 120 fault episodes (70%), 111 obligations, computed on the completion tags the binary produced, with no modified binary run and no competitor; the ten no-fault controls lose nothing and the ten foreign-identity cases are reported separately. The finite projection finds the same loss in 260 of 772 states.

Abstract	Where	Precise statement and caveat
"Registry, CI, replay and scale experiments include google-cloud-node and selected mid-release commit conflicts."	Section 10.4; Section 10.3 (replay, orphan and push scenarios); Section 10.5	Registry: the two topologies of the companion campaigns. CI: the three-repetition fixture. Replay: five releases each of `google-cloud-node` (36 of 42 exact) and `conventional-changelog` (7 of 64) under reconstructed layouts. Mid-release conflicts: the E5 cells of the 1.8.2 campaign, a clean and a conflicting concurrent push on selected interleavings (12 and 14 checks), not a systematic sweep. Scale: the eight timing cells of Table 8 on a capped, contended host.
"Module suites include scanner/writer fixtures from real projects; release CI requires the full suite regardless of change scope and checks binaries before publication and installations afterwards."	Section F11; the product's release workflow; evidence `experiments/distributed-20260922/evidence/product-gates/`	Section F11 maps this report's results to the integration tests behind them. The rest is the product's release policy as written in `.github/workflows/release.yml` at `252216ba`: a "Full suite" job runs every package's tests on every release, the publication job depends on it, and the workflow's header describes the binary build, the publication and the install check as gates on one another. The release candidate's gate logs of 22 September (the sharded suite 6 of 6 on `272772cb`; integration-only coverage 94.0% against a 95% gate, with a dispatch input to lower it; the TinyGo gate skipped by dispatch input) are copied under `product-gates/`. This report did not audit the workflow or the scanner and writer fixtures; it reports the policy as read.
"Native Windows and distributed execution are outside these measurements. Linux and LLVM results will be presented in the report."	Section 10.6; Section 10.7	Presented in Section 10.6: kernel farm round 1 (every candidate rejected), round 2 (hexagon 95.89 and 92.22 s against 111, m68k_nommu 115.61 and 115.48 s against 128, after one rejected candidate; the single node 122.62 and 138.01 s) and the LLVM matrix (27:48.94 against 41:12.02, once per side; the negative case 26:47.69 against 20:16). One repetition per side, asymmetric hardware, no general speedup claimed. Native Windows remains unmeasured.

References

[1] *Conventional Commits: Monorepo Extension (CCME) 3.1.0-rc.2.* `specs/ccme-spec/SPEC.md` at `252216ba`, `github.com/yohimik/dispat`, 21 September 2026. The current normative subject, cited throughout as SPEC. GPL-3.0-or-later.

[2] *CCME design history and changelog.* `specs/ccme-spec/DESIGN-HISTORY.md` and `CHANGELOG.md` at `252216ba`. Dated entries stating what the engine implements and where it departs; the entries of 21 and 22 September 2026 are quoted in Sections F10 and 10.

[3] *CCME 2.0.0*. `released CCME 2.0.0` at `d3ce8ae6`. The version the earlier editions of this report formalised; the historical audit uses the `CCME 1.0.0 snapshot` at `713f1a0b`.

[4] *CCME 3.0.1*. `released CCME 3.0.1`. Corrected algorithm and cost guidance between 2.0.0 and the current text.

[5] *dispat*: a release engine for polyglot monorepositories and polyrepositories. `github.com/yohimik/dispat`, commit `252216ba`; latest released candidate `services/dispat/v1.11.0-rc.3`. The reference implementation; the release computation is `services/dispat/internal/plan`, the execution profile `services/dispat/internal/execution`.

[6] *dispat integration test plan and requirement matrix*. `tests/integration/docs/test-plan.md` and `qa-requirements.md` at `252216ba`. Sixty goals, the claim each integration test proves, the regression and bug fences; the source of the integration rows of Section F11.

[7] *Conventional Commits 1.0.0*. `conventionalcommits.org`. CCME is a strict superset.

[8] T. Preston-Werner. *Semantic Versioning 2.0.0*. `semver.org`. Supplies the version precedence of Definition 7.

[9] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119, IETF, 1997. The requirement levels SPEC is written in.

[10] H. Garcia-Molina and K. Salem. *Sagas*. In Proceedings of the ACM SIGMOD International Conference on Management of Data, 1987. The pattern the publish protocol of Section 4 implements, with git tags as the log.

[11] A. B. Kahn. *Topological sorting of large networks*. Communications of the ACM 5(11), 1962. The publish order of Proposition 45.

[12] R. L. Graham. *Bounds for certain multiprocessing anomalies*. Bell System Technical Journal 45(9), pages 1563 to 1581, 1966. The scheduling anomaly of Example 63.

[13] R. L. Graham. *Bounds on multiprocessing timing anomalies*. SIAM Journal on Applied Mathematics 17(2), pages 416 to 429, 1969. The list-scheduling bound of Proposition F73.

[14] S. Chacon and B. Straub. *Pro Git*, 2nd edition. Apress, 2014. The commit DAG, reachability and tags of Definitions 1 and 8.

[15] G. Roeck. *kerneltests.org*: the Linux kernel build and boot test farm, `https://kerneltests.org/`, builders `upstream-<arch>-master` builds 330 (hexagon), 331 (m68k_nommu) and 332 (alpha) for v7.3-rc4, read on 22 September 2026 and retained in `dispat/gce/parity-20260922-reference.json` of the fork `github.com/yohimik/linux-build-test`. The public reference times of Section 10.

[16] *linux-build-test*, fork of G. Roeck's farm configuration with the dispat prototype. `github.com/yohimik/linux-build-test`, branch `codex/kernel-parity` at `c699bf3d`; reports `dispat/gce/REPORT.md`, `PARITY-20260922.md` and `PARITY-20260922-round2.md`; oracle `dispat/compare.py`.

[17] *llvm-project*, fork with the dispat recipe. `github.com/yohimik/llvm-project`, branch `dispat-gce` at `952c2de4`; report `.ci/dispat/gce/REPORT.md`; recipe `.ci/dispat/build.py`; oracle `.ci/dispat/gce/compare.py`.

[18] *kernel.org cross toolchains.* `https://mirrors.edge.kernel.org/pub/tools/crosstool/`, release 16.2.0 (GCC 16.2.0, binutils 2.47) for x86_64 and arm64 hosts, with published SHA-256 sums; the alpha and m68k compilers of the kernel farm.

[19] *Google Compute Engine machine families.* `https://cloud.google.com/compute/docs/machine-resource`, read 22 September 2026: `c3-standard-22`, `t2d-standard-8`, `n2-custom-14-49152` and `e2-standard-4`, the shapes of Section 10.

[20] Semyon F. (yohimik). *HOL Guard pull request 2813: opt-in Dispat release-start review.* Open, unmerged pull request, revision `132bb5d105e1234b9f81fddf69267ec488e2990a`, 6 September 2026.